

PART 2 – NEXT STEPS IN 3D MODELING

The second part of the primer builds on the model started in the previous section. The model is used to introduce additional capabilities in *FlexSim* and illustrate how they are used to represent operations systems. In addition to introducing new modeling objects such as task executors and conveyors, this part introduces some of the modeling *tools* that are available in *FlexSim*.

The *FlexSim* capabilities included in this part are:

- importing layouts and modifying object graphics,
- routing flowitems between objects by various criteria,
- incorporating dynamic objects, referred to as task executors into a simulation model (e.g. operators and transporters).
- transferring items between objects via conveyor objects,
- managing model data through the use of global tables, and
- incorporating planned and unplanned downtime on objects through time tables and reliability, respectively.

1 CHANGING 3D GRAPHICS

Although excellent simulation models can be created with the default graphic for the objects in *FlexSim*, there are two main reasons to become acquainted with the basics of importing graphics.

1. Using images that correspond to the domain that is being modeled helps users better relate to the model. For example, a Processor object would typically be used to represent the operation of a MRI machine in a healthcare project. While the functional aspects of the Processor represents the MRI, the default shape is a machine found in manufacturing. Doctors, nurses, technicians, and administrators would understandably have a problem relating to the manufacturing image representing an MRI machine.
2. Building models on a layout makes setting sizes, locations, and distances much easier. This is especially important when considering the transport of items between objects whether on conveyors or by task executors (operators, fork trucks, etc.) since distance affects system performance.

This primer only considers the very basics of working with graphics. For more information, please see *FlexSim's* User Manual.



Begin with the basic model from the previous section, named *Primer_1-2*, and **Save As** *Primer_2-1*. Basically make a copy of the model so that it can be customized and the original model is retained.

1.1 Importing a layout

There are several ways to import a layout into a *FlexSim* model, including importing an *AutoCAD* file. For the primer, the layout is just a two-dimensional graphics file, named *Layout.prg*. It was created in *Microsoft Office*, but could have been created in any drawing program. It is imported into *FlexSim* as a texture on the **Plane** object, which is found in the **Visual** section of the object Library.

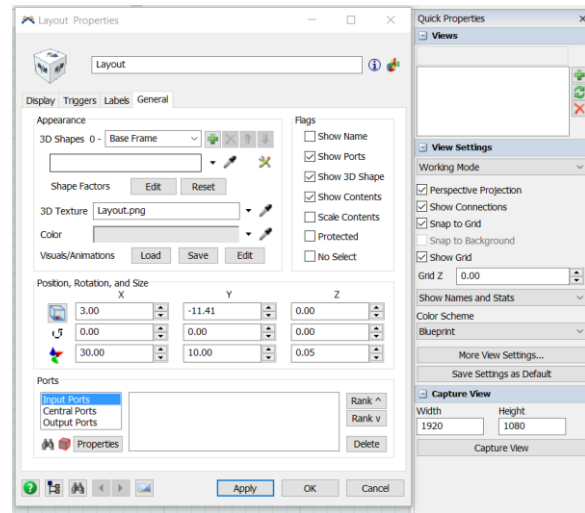
The object interface is shown below. To import the layout:

- Drag out the Plane object to the modeling surface in the 3D view.
- Change the name of the Visual object from Plane to Layout.
- By default, the plane is a solid-colored object. To show the layout, select the **Filename** drop-down menu in the **Texture** section of the Display tab and select **Browse**, at the top of the menu. The Browse option opens Windows Explorer. Use it to locate the layout file on your computer. Note that once the graphic is imported into *FlexSim*, it will appear as part of the **Browse** menu – therefore, if you use the graphic again in the model, you would not need to import it from your computer.

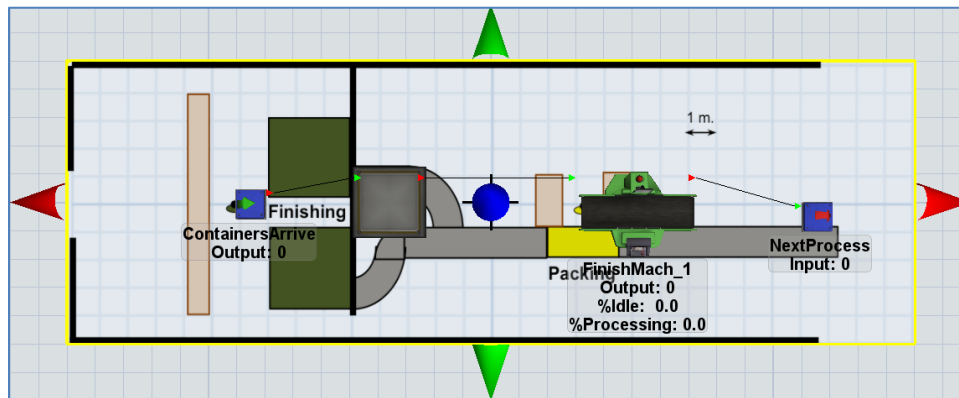


It is good practice to place all supporting model resource files, e.g. graphics and spreadsheet files, for folders containing the files in the same computer directory or folder as the *FlexSim* model. This makes transferring the model to other computers much easier.

- Size the layout object to $x=30$, $y=10$, $z=0.05$ since the workspace is 30 meters by 10 meters. As shown in the figure to the right, this can be done either through the General tab of the visual object or via the Quick Properties window.



- Drag the layout so that it is centered at the origin as shown below. The location values should be $x = 0.0$, $y = 0.0$, and $z = 0.0$.

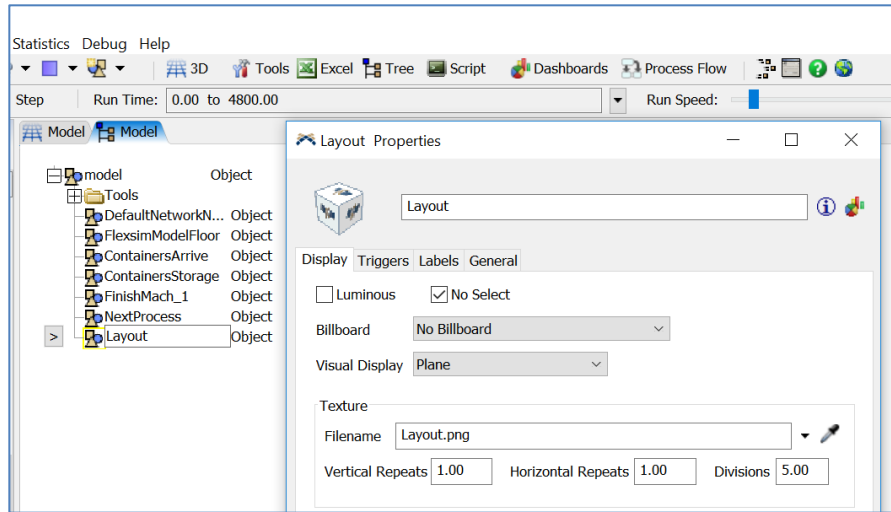


As other objects are placed onto the layout, the layout object might accidentally get selected and moved or other objects may become imbedded in the plane (a useful feature for building hierarchical models, but not in general). In order to avoid these issues,

- The property **No Select** should be checked. It is found both on the Display and General tabs of the Visual object named Layout.

A drawback of enabling the **No Select** property is that, as the name indicates, the object cannot be selected in the 3D view. It must be opened from the **Tree** – the underlying hierarchical data structure in *FlexSim*. Working with the Tree is an advanced topic, but is briefly introduced here.

The Tree is where all of the model data is stored in *FlexSim*. Each object interface is just a link to data in the Tree. The Tree for this model is shown below. It is accessed from the Main Menu icon labeled 'Tree'. To open an object, just double click on the object in the Tree view. This is the same interface that is obtained by double clicking on the object in the 3D view.



Placing the objects in their proper location on the layout in the 3D view is explained in the next section.



When placing objects, it is usually better to work:

- in 2D rather than 3D (right click anywhere on the modeling surface, select **View**, and then **Reset View**).
- **not** in the default perspective view (right click anywhere on the modeling surface and uncheck the **Perspective Projection** box in the **View Settings** section of Quick Properties window).

1.2 Changing object graphics

Before placing the existing objects on the layout, a few of their properties are changed, i.e., their size and 3D shape.

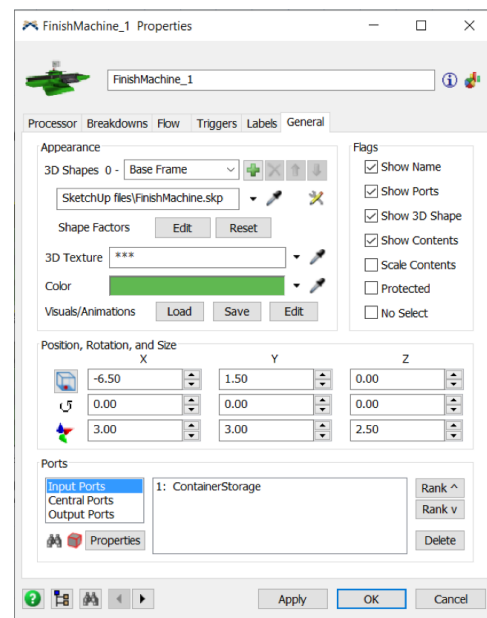
While a 3D shape can be created to represent any system object, many such shapes are readily available. For example, 3D Warehouse (<https://3dwarehouse.sketchup.com>) contains millions of 3D objects that have been created in the 3D modeling software *SketchUp*. Here are a few comments about *SketchUp* and using shapes developed in *SketchUp* in *FlexSim* simulation models.

- Shapes from 3D Warehouse can be downloaded for free and used in *FlexSim*.
- *SketchUp* 3D modeling software is free and can be used to modify any shape downloaded from 3D Warehouse or to build a custom 3D shape object from scratch.
- *SketchUp* is now a web-based application, but older downloadable versions for Mac OS and Windows are available at the SketchUp Help Center (<https://help.sketchup.com/en/article/60107>). *SketchUp Make 2017* is suggested. It is the free version; *SketchUp Pro* has more features, but requires a fee.
- Since file formats change often, it is best not to use the latest format since it may not be compatible with *FlexSim*. Therefore, downloads should be *SketchUp* file version 15 or earlier.
- The size of the *SketchUp* model, especially the number of polygons, affects the *FlexSim* model size and its run time. Therefore, very complex graphics should be avoided unless they are really needed and the model is run on a powerful computer.

Finish Machine (Processor)

The shape of the finish machine is changed from the default graphic to the *SketchUp* file named FinishMachine.skp. As shown in the figure to the right:

- Associate the *SketchUp* file with the *FlexSim* object FinishMachine_1 by selecting **Browse** in the drop-down menu under **3d Shapes** in the **Appearance** section of the object's General tab. The path SketchUp files\FinishMachine.skp indicates that the graphics file is in a graphics file named "SketchUp files," which is the same location as the *FlexSim* model. By default, graphics media are imbedded within a model so that the model may be copied without the supporting external graphics files.
- Change the object size properties to $x = 3$, $y = 3$, $z = 2.5$, all values are in meters; i.e. each machine is three meters square and 2.5 meters high.
- Change the location properties to $x = -6.5$, $y = 1.5$, $z = 0$. The object could have been placed in the correct location by moving it manually on the layout



Another finishing machine will likely be needed since the average input rate is equal to the average processing rate; therefore, it is a good time to add it now.

- Copy FinishMach_1 using Cntl-C shortcut key combination, then click on the layout near where the machine will be located, and paste using the Cntl-V keyboard shortcut.

- Rename the copied object as FinishMach_2 and place it in the proper location on the layout ($x = -6.50$, $y = 2.40$, and $z = 0.0$).
- Make the flow connections: an A-connection from the Queue named ContainerStorage to the finish machine and another A-connection from the finish machine to the Sink named NextProcess.



When copying and pasting objects, it is best to copy (Cntl-C), then click on the modeling surface approximately where the new object is to be located, and then paste the copied object (Cntl-V). If you don't click on the modeling surface, the pasted object may become a subset of the copied object. This is a good feature in some cases, especially where you want to build submodels, but that is a more advanced topic.

Container Storage (Queue)

Next, the Queue or ContainerStorage object is customized. Its shape is changed to a Plane, which is a basic 3D shape that comes with *FlexSim*. It is located on top of the storage area in the imported layout.

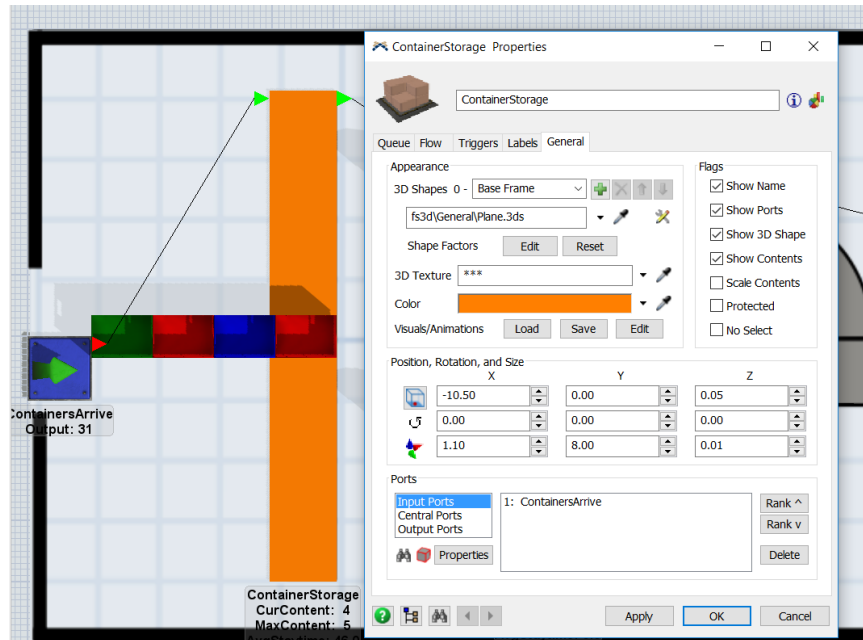
- As shown in the figure below, for the **3D Shapes** property in the **Appearance** section of the Queue's General tab, select **Browse ...** from the drop-down menu in order to find *FlexSim*'s fs3d folder. This folder contains a variety of graphics files and is located in the *FlexSim* folder that is within the Program Files folder, typically on the computer's C: drive.

Change the following properties on the General tab or in the Quick Properties panel when the object is selected.

- Using the drop-down menu, change the object's color to orange.
- Set the object's location to $x = -10.5$, $y = 0.0$, $z = 0.05$. Note the object is located a bit above the base of the modeling surface ($z = 0.05$, not $z = 0$) – this is so it can be seen above the layout object.
- Set the object's size to $x = 1.1$, $y = 8$, and $z = 0.01$. Note that the x value is set slightly larger than the container size.



If a Queue's size is less than or equal to the size of an item that is trying to fit onto the Queue, it will not fit; as a result, *FlexSim* will position it outside of the Queue. The logic will not be affected, but it will look strange.

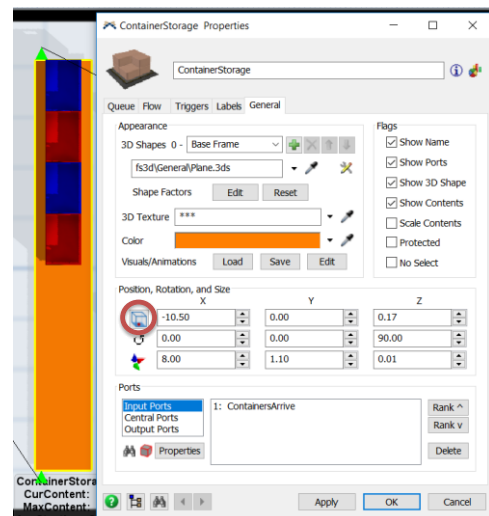


The above changes should place the object over the storage area on the imported layout.

When the model is run, as shown in the figure above, the containers do not conform to the shape of the storage area. This is because only the general shape of the object has been changed; logically items still queue in a horizontal line from the front of the object to the back of the object and beyond. Basically items line up from right to left.

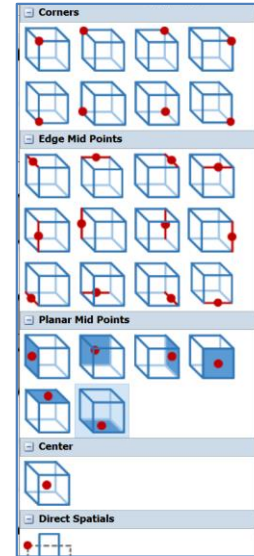
To correct this, make the following changes on the Queue's General tab. The revised properties are shown in the figure to the right. The figure also shows the effect on the model – containers now line up the same way they do in the real system.

- Rotate the Queue 90 degrees about the z axis so that it is vertically oriented and not horizontal. Set the z rotation to 90.0.
- Since the item is now rotated, reverse the sizes in the x and y directions; i.e., x = 8.00 and y = 1.10.
- Since the size and rotation have changed, the object needs to be repositioned; i.e., set the locations to x = -10.5 and y = 0.00.



This change may appear to be only for appearances, but in this case, the queue orientation will affect estimated system performance. Later, when an operator is added to move containers from the storage area to the machines, the location of the containers to be moved will affect the operator's travel time and thus performance.

You might notice a difference in the object's location property values between the General tab on the object and the Quick Properties window. This is due to the availability of different possible reference points in *FlexSim*. The possible reference points are shown in the figure to the right. The reference options are accessed by clicking on the small box-like icon to the left of the location values on the General tab, as indicated by the red circle on the interface in the previous figure.



ContainersArrive (Source) and NextProcess (Sink)

The Source and Sink shapes and sizes are not changed. They are just moved outside of the layout area – the Source is moved off to the left and the Sink is moved off to the right, both outside of the layout.



If you haven't already done so, save the model. Recall, it is good practice to save often.

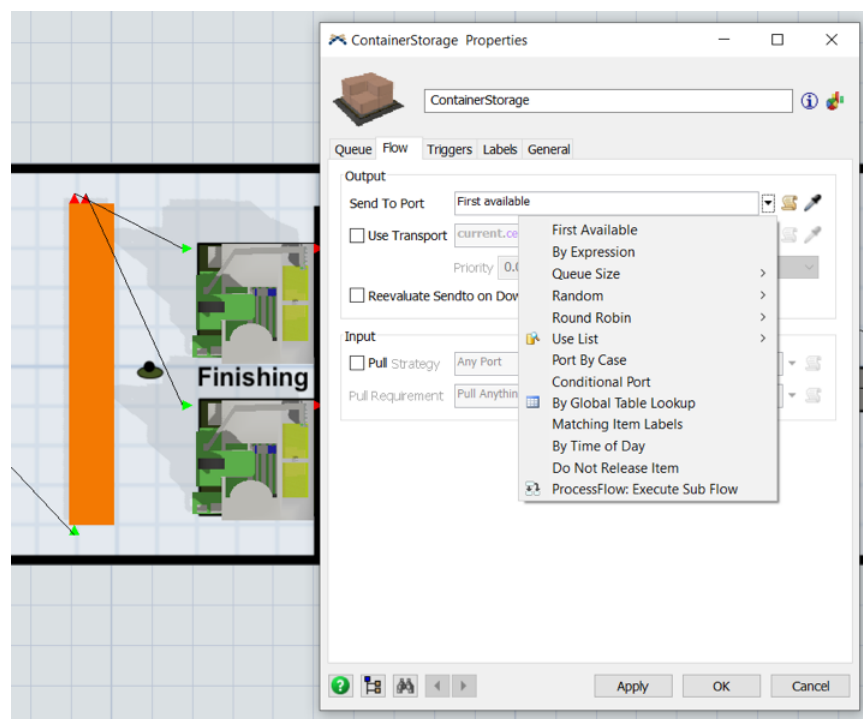
2 ALTERNATIVE ROUTINGS

Routings are a key part of modeling process flows. As items flow through a system they may follow different paths due to the current condition or state of the system. For example, different paths may be followed depending on the type of the object, the availability of a resource, the perceived waiting time to secure a resource, etc. Paths may also change randomly, such as a customer choosing between two available servers.

Until the previous step, all items (containers) followed the same routing path – they all flowed through the four basic objects in sequence, Source to Queue to Processor to Sink. But now there are two Processors in parallel; i.e., an item can go to either one. Thus, there is a routing decision at the Queue. Also, since the capacity of the Queue is limited, there is a routing decision at the Source – either go to the Queue or somewhere else if the Queue currently has no remaining capacity.

2.1 Choosing a route based on availability

The Queue for storing containers awaiting finishing has the option of sending a container to either finishing machine 1 or 2. For now, the default setting, FirstAvailable is used. The routing logic is controlled at the Send To Port trigger on the Flow tab, as shown in the figure below. Note in the figure that there are many available routing options to move items from the Queue to one of the machines.



The First Available option works as follows. When an item is ready to leave the Queue, it checks to see if the downstream object that is connected to Port 1 is available to receive the item. If it is, then the item is sent – in this case, the item leaves the Queue and arrives at the first machine in zero simulated time. Recall that by default, it takes no time to move from object to object in a model. If the first machine is not available, then the Queue checks to see if the downstream object that is connected to Port 2 is available to receive the item. If it is, then the item is sent – in this case, the item leaves the Queue and arrives at the second machine in zero simulated

time. If both machines cannot accept the item, then the item waits in the Queue until one of the downstream machines becomes available. Note that the port connections between objects drives the routing decision by setting the sequence for checking machines for availability.

Using the FirstAvailable routing logic usually results in the object connected to the first port receiving more items since it is always checked first. Similarly, the object connected to the last output port will receive the fewest items.

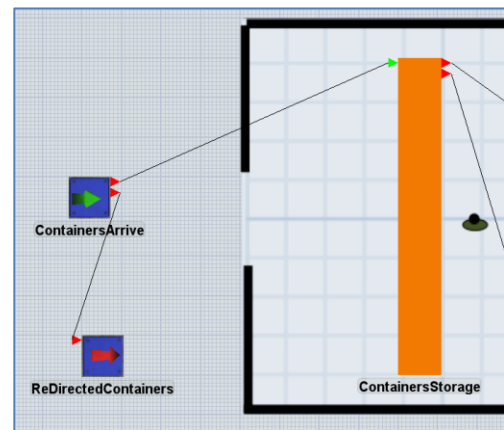
2.2 Choosing a route based on current system conditions

The container storage area has a limited capacity (Maximum Content's property value on the Queue is 50). Therefore, provisions must be made for the condition when the Queue is at capacity and cannot accept any more containers. In this case, containers will be diverted to a second Sink, named ReDirectedContainers, if the Queue is full.

Implement this in the model as follows and as shown in the figure to the right.

- Drag out a Sink object from the Object Library onto the modeling surface and name it **ReDirectedContainers**.
- A-connect the Source to the new Sink.

As shown in the figure to the right, there are now two output port connections from the Source, one going to the Queue and one going to the second Sink that was just created. As with the case above, the default **FirstAvailable** is used so that items go out Port 1 to the Queue if there is available capacity and go out Port 2 to the Sink if the Queue is full.



Note that if the connections are reversed the model would behave differently. That is, if the Source and Sink are connected through Port 1 and the Source and Queue through Port 2, then all items would go to the Sink and none to the Queue since a Sink is usually always available and has no capacity limits. Therefore, it is important to be careful on the order in which objects are connected.



If you haven't already done so, save the model. Recall, it is good practice to save often.

3 TASK EXECUTERS

Task executers (TEs) are mobile or dynamic resources – they move about in a model, typically moving flowitems between objects. There is a section in the Object Library, right below Fixed Resources, that contains different types of TEs, including Operators, Transporters, Cranes, Robots, etc. All of these have the same functionality and basically differ in the way they move; e.g. Operators can move in any direction (x, y, and z), whereas Elevators can only move vertically (z direction).

TEs can be used to represent complex behaviors, but only the most basic properties and a single type of TE, the Operator, are covered here. Prior to discussing how to implement TEs in a *FlexSim* model, a few general concepts are introduced.



[Section 6.1] **Mobile resource objects (task executers)**

3.1 Basic task executer concepts

TEs have speeds (and accelerations); therefore, the location of model objects and distances between them now become very important.

As the name indicates TEs execute tasks. Sets of tasks for performing a specific operation, called a **task sequence**, are passed to TEs, typically from fixed resources (sources, processors, etc.). *FlexSim* includes default task sequences, but as in most things in *FlexSim*, they can be customized. For now, only default task sequences are considered. For transporting items a TE needs to move items from one object (FromObj) to another object (ToObj). To do so, by default, a TE executes the following sequence that is sent to the TE from the FromObj.

Travel from current location to FromObj
Load item from FromObj
Travel from FromObj to ToObj
Unload item to ToObj

Tasks other than travel, load, and unload are available in *FlexSim*. However, developing custom task sequences is an advanced topic and is covered later in this primer.

Fixed resource objects communicate with TEs via special connections, referred to either as a Center-connection or S-connection. It is called a Center-connection because the connection is made between the center ports of the two objects. It is called an S-connection because the connection between the two objects is made by holding down the S key while dragging the mouse between the two objects.

Objects have center ports in addition to the input and output ports that were introduced earlier. Center ports are not for item flow, but are used for communications between objects. Communications can be bidirectional between the objects; this is in contrast to the unidirectional flow between input and output port connections (A-connections).

A TE will oftentimes receive more requests to perform task sequences than can be met at that time. For example, a TE may receive a request while performing a task sequence. Therefore, TEs can queue requests to

perform task sequences and can use different means to process items in the task queue, e.g. first-in, first-out or priorities.

TEs not only receive tasks, but can send them to other TEs. Thus, TEs are like working managers – they will carryout a task-sequence request unless they are busy when they receive the request. If they are busy when they receive the request, they can send it to an available TE. If all associated TEs are busy, the receiving TE puts the task-sequence request into its work queue. To implement this, decide which TE is the working manager and connect it to all associated TEs via A-connections (from the working manager TE to the associated TEs).

If all TEs are considered the same and no one is the manager, then a Dispatcher object is used. The only function of the Dispatcher is to allocate work and maintain the work queue for all of the associated TEs. In this case, the fixed objects communicate with the Dispatcher and not directly with the individual TEs. Therefore, the fixed objects are connected to the Dispatcher with S-connections through their center ports and the Dispatcher is connected to each TE with an A-connection (from Dispatcher to TE).

By default, a TE travels between objects in a straight-line path, traveling the shortest distance between two objects. A TE's default travel path does not consider other objects in its path. However, there are several alternative means to control the travel path of a TE – one, using path networks, is discussed in a later sub-section and another approach, using the A* algorithm, is discussed in a later section once the conveyors and the packing station are included in the model.

3.2 Adding a finish operator to the model

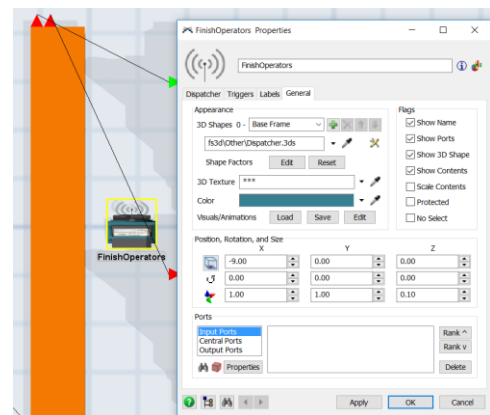
Since it is possible that more than one finish operator will be needed, use both the Dispatcher and Operator objects.



Begin with the basic model from the previous section, named Primer_2-1, and **Save As** Primer_2-2. Basically make a copy of the model so that it can be customized and the original model is retained.

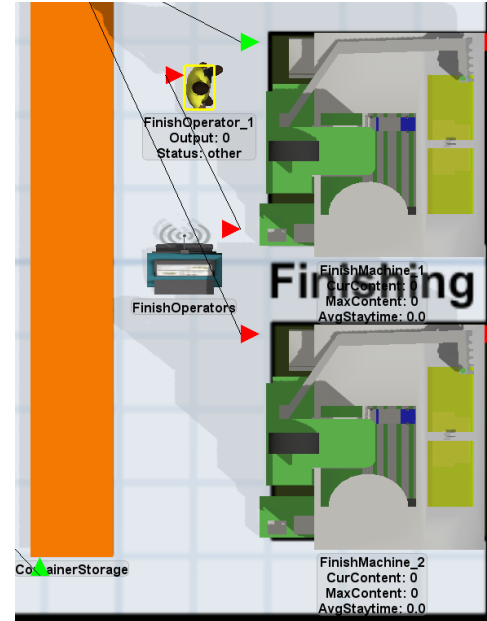
- Drag out a Dispatcher object from the Task Executors section of the Object Library to the modeling surface. As shown in the figure to the right:

- Name the object **FinishOperators**.
- Locate the object where the person symbol appears on the layout in the finish area ($x = -9.0$, $y = 0.0$, $z = 0.0$). It can be placed anywhere, but this is a convenient location in this example.
- Resize the object using the size settings $x = 1.0$, $y = 1.0$, and $z = 0.1$. This minimize the object's presence since the Dispatcher object is not a real entity in the system.



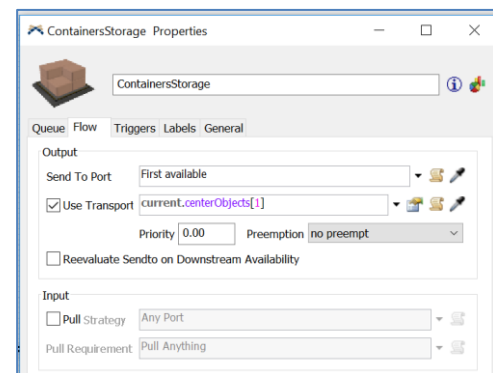
The Queue makes requests of the Dispatcher, i.e. sends task sequences, to move items from the Queue to one of the Processors. The Dispatcher then sends the request (task sequence) to one or more Operators that it manages. Implement this aspect in the model as described below and as shown in the figure below and to the right.

- Make an S-connection between the Dispatcher and the ContainerStorage Queue by holding down the S key, selecting the Dispatcher so that is highlighted by a yellow box, dragging the mouse to the Queue, and releasing once the Queue is highlighted with a yellow box. As shown in the figure to the right, there should be a connection between the two objects' center ports. When making center-port connections, the order of connections is not important since it involves bi-directional communications; i.e., the connection can be made from the TE to the fixed object or from the fixed object to the TE.
- Drag out an Operator object from the Task Executors section of the Object Library to the modeling surface. At the moment, it can be placed anywhere.
- Name the object **FinishOperator_1**
- Make an A-connection from Dispatcher to FinishOperator_1 by holding down the A key, selecting the Dispatcher so that is highlighted by a yellow box, dragging the mouse to the finish operator, and releasing it once the Operator is highlighted with a yellow box. As shown in the figure to the right, there should be a connection between the Dispatcher's output port and the Operator's input port. Make sure the connection is from the Dispatcher to the Operator since it is a one-way flow of information.



Now that all of the objects can communicate, the Queue needs to be able to send a task sequence to the Dispatcher telling it what to do; in this case, the Queue requests the TE to transport an item from the Queue to an available finish machine. This is accomplished as follows and as shown in the figure.

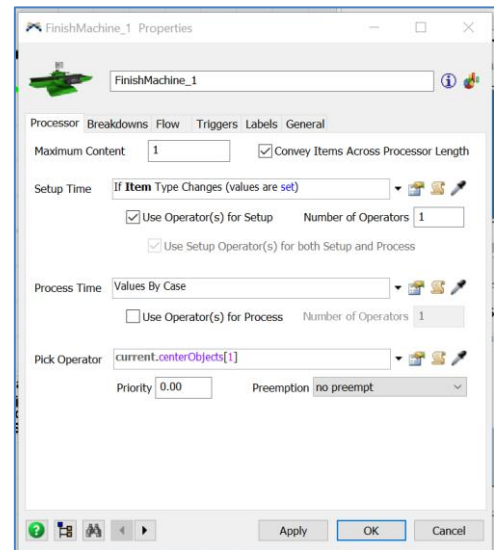
- On the Flow tab of the Queue object (ContainerStorage), check the **Use Transport** box. By default, the task sequence goes to whatever object is connected to the Queue's center port; in this case, that is the Dispatcher. This action is accomplished by the default **Use Transport** property value, which is the `current.centerObjects[1]` FlexSim command. This command is interpreted as the object connected to the current object's center port. In this case, the current object is the Queue and the object connected to the first center port is the Dispatcher.



Once the Dispatcher receives the task sequence it sends it to the Operator when the Operator is available.

The Operator also needs to perform the setup operation at each finish machine. This is done similar to the process just described. As shown in the figure to the right and as described below, modify each Processor as follows.

- On the Process tab, check the **Use Operator for Setup** box that is below the Setup Time property..
- The **Pick Operator** property, near the bottom of the Process tab, can remain the default. Each Processor will use the object connected to its first Center port, which will be the Dispatcher.
- Make a S- or Center connection between each Processor and the Dispatcher.

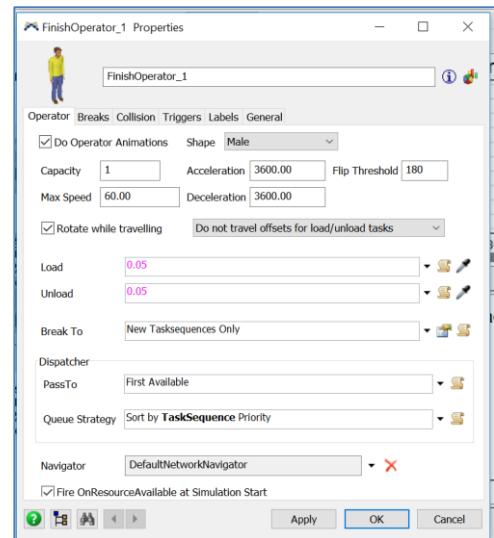


Since the performance of the system depends on speeds and distances, Operator and other Task Executer properties are important. The default value for an Operator's speed is 120 meters/minute. This seems a bit fast for this operation since the average walking speed is about 84 meters/minute. Therefore, use a value somewhat slower than average walking speed, 60 meter/minute.

- As shown in the figure to the right, change the **Max Speed** value, on the Operator tab on FinishOperator_1, from the default of 120 to 60.

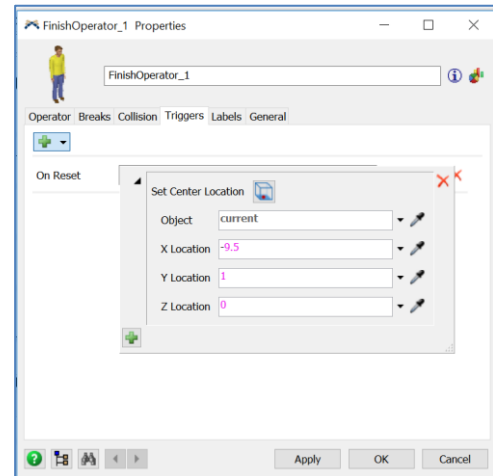
Also, consider for a transport task sequence the time it takes to load a flowitem at the originating object and unload a flowtime at the destination object. The default time is 0.0. For this example, change each to 3 seconds.

- Also on the tab as above, change the **Load** and **Unload** property values to 0.05 minutes (3 seconds).



Since the Operator object is mobile it can be in any location on the modeling surface when a running model is stopped. For consistency in comparisons and analyses, it is good practice to have the Operator start at the same location each time a model is **Reset** and **Run**. Therefore, in this case, set the location of the FinishOperator_1 to a location near finish machine 1 and the container queue.

- As shown in the figure to the right, on the Operator object's Trigger tab, add an **On Reset** trigger and then select the **Set Location** option. Set the location values to $x = -9.5$, $y = 1$, $z = 0$. The Operator moves to this location whenever the **Reset** button is pushed.



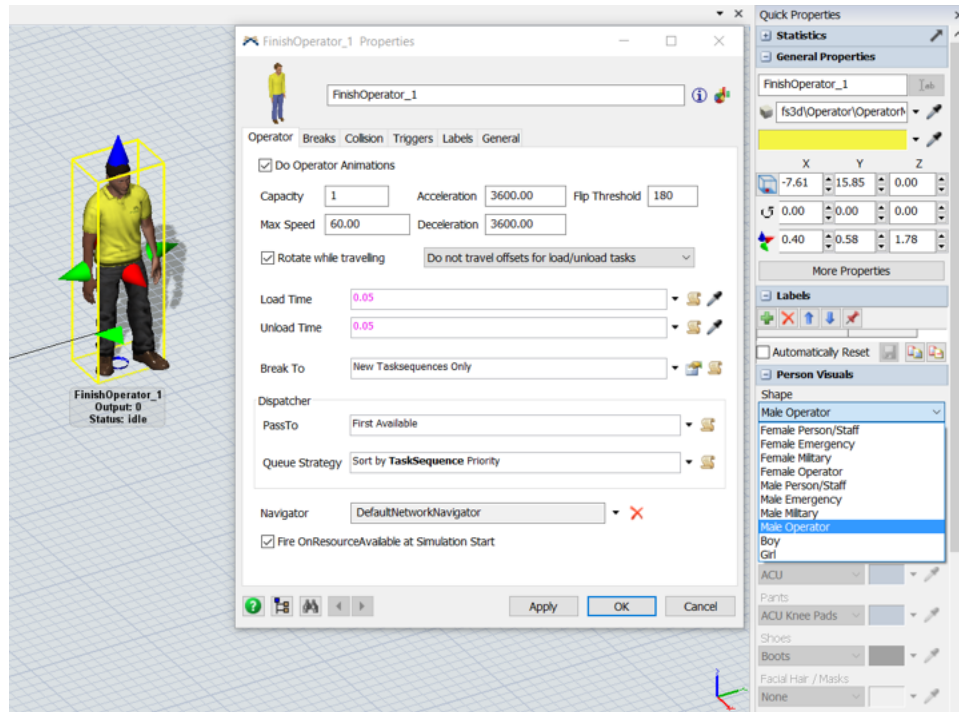
By default, the last task in the Operator's transport task sequence is Unload; therefore, it will become available for another task once the Unload operation is complete. If there are no other tasks waiting, the Operator will remain at the Unload location until it receives another task. For now, this is okay, other options can be considered later.



If you haven't already done so, save the model. Recall, it is good practice to save often.

- **Reset** and **Run** the model. Note that the Operator carries containers between the Queue and Processors and remains at the Processor to perform the setup operation. You may need to slow down the model's speed to see this (e.g. Run Speed = 0.1) or stop the model as a transport is taking place. Also, note that the Operator returns to the same location when the model is reset.

Another feature of the Operator object is that its form is very customizable. The form or shape of the default Operator object is that of a **Male Operator**. The value of the **Shape** property is shown in the Quick Properties window when an object is selected. Note that in the figure below the only customizable feature of the **Male Operator** (in the Quick Properties window) is the **Skin Texture** property; all other properties, e.g. **Head**, **Hair/Hat**, **Shirt**, **Pants** are all grayed out. This is also true of the **Female Operator**. However, all other **Shape** values, as shown in the drop-down menu in the figure below, are customizable, such as **Female Person/Staff**, **Female Emergency**, ... **Boy**, **Girl**. While changing the shape/form of the Operator object does not affect the system's behavior, it does help those not familiar with simulation better identify with, and oftentimes have more confidence in, the model. At this point in the primer, no changes are made to the Operator's shape; the default Operator properties are sufficient for the problem domain.



Another feature of Operator object, and most other objects in *FlexSim* as well, is that custom **animations** can be defined. Animations are a sequence of object movements that are triggered as a simulation runs. For the Operator, the default animations are walking and carrying a flowitem. Animations are created, edited, imported, and exported via the Animation Editor, which is accessed via the **Visuals/Animations** property on the object's General tab. At this point in the primer, no changes are made to the Operator's animations; the default Operator properties are sufficient for the problem domain. However, animations are introduced via an example later in the primer.

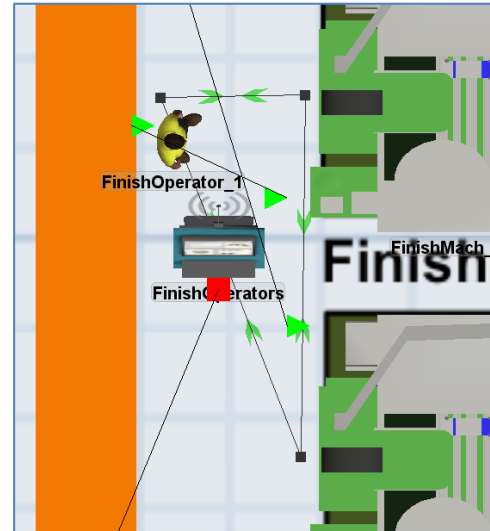
One visual that is quite useful, and is easy to control, is an object's color. The color of an object can be changed through drop-down menu options on a number of triggers. For the Operator object, the shirt color is what is changed. This can either be done statically via the **Color** property on the General tab or via triggers, e.g. set the color of a flowitem when it is created based on its type or change an object's color when it is down or not available. Using colors is a very effective way of testing and validating a model and is used extensively in this primer.

3.3 Controlling task executor travel with path networks

By default, when TEs travel between objects in a model, they use straight-line paths and are not inhibited by other objects. To better match reality, and to provide more realistic performance measures, TEs are constrained to defined paths that are formed by what is referred to as a **path network**, which is similar to a roadway. A path is defined by connecting a series of nodes, referred to as **network nodes**. The nodes are fixed objects and are located in the Travel Networks section of the Object Library, just below the Task Executor section.

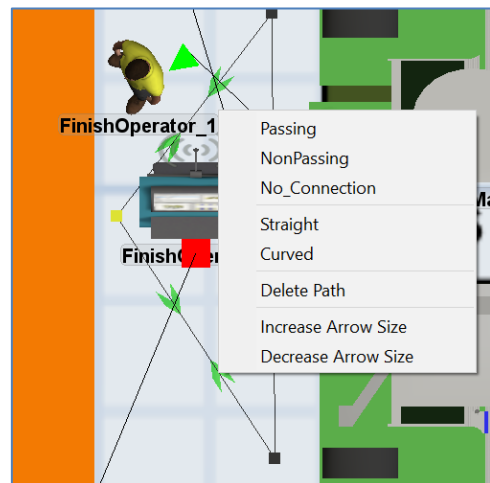
Only a simple network is constructed here since later, once more objects are included in the model, a different means of controlling TE travel will be used. For now, travel will be controlled with three nodes, one at each finish machine and one at the queue (ContainerStorage).

- Drag three Network Nodes from the Object Library to the modeling surface and place them in the locations shown in the figure to the right. Network Nodes are small square objects.
- In order to get exact placement of the Nodes, you may want to uncheck the **Snap To Grid** box under **View Settings** in the Quick Properties window. Otherwise, nodes will be placed on the closest grid point. Recall this interface is accessed when clicking anywhere on the modeling surface where there is not an object.
- Since it is good practice to name objects, name the Nodes: **nn_ContainerStorage**, **nn_FinMach_1**, and **nn_FinMac_2**. No other properties need to be modified.
- The path segments, oftentimes referred to as *edges*, are formed by connecting the Nodes with A-connections, as shown in the figure to the right,



The green arrows on the edges denote the permitted direction of travel. By default, the edges are bidirectional. As shown in the figure to the right, right clicking on an arrow provides the following options:

- **Passing** permits multiple TEs on the same path to pass or move past each other. This is the default.
- **NonPassing** restricts faster TEs from passing a slower TE on the same path.
- **No_Connection** restricts the direction of travel.
- **Straight** declares edges, or travel paths between nodes, to be straight lines. This is the default.
- **Curved** declares edges, or travel paths between nodes, to be curves that can be adjusted.
- **Delete Path** eliminates an edge between nodes.
- **Increase Arrow Size** makes the directional arrows larger.
- **Decrease Arrow Size** makes the directional arrows smaller.



In order for the network to function, fixed objects must be associated with network Nodes. This is so that a TE knows where on the network an object is located. For example, if an Operator is to travel to a Queue in order to pick up an item, and the TE must stay on the network path, it must know which node to travel to in order to reach the Queue.

Objects are associated with Nodes by making an A-connection between them. When connected, a blue line denotes the connection, as shown in the figure to the right.

- A single object can be associated with multiple Nodes; e.g., if an object can be accessed from more than one side or if an object is associated with multiple networks.
 - Similarly, a single network Node can be associated with multiple objects, e.g. if a TE interacts with a group of objects from a common point.
- Connect each network node to its associated object; i.e., there are three connections. For example, one connection is from Network Node nn_FinMach_1 to Processor FinishMachine_1.

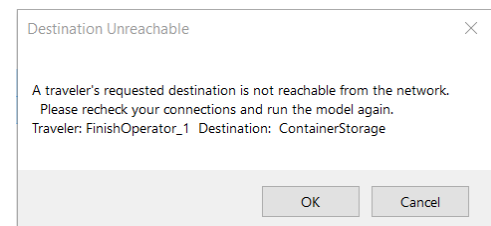
To use a network, each TE must be associated with the network. This is accomplished by an A-connection between the TE and only one of the Nodes on the network. This is considered the TE's "home" Node – where it will be located when the model is Reset. The resulting connection between the TE and the Node is indicated by a red line, as shown in the figure to the right.

- Connect the Operator to the Network Node associated with the Queue, ContainerStorage.



When the model is run, an error will result if any of the objects where the TE needs to perform a task is not associated with a network node.

An example error message is shown to the right. In this case, the Queue ContainerStorage has not been connected to its associated Network Node because the finish operator needs to travel there in order to pick up a container, but the Operator doesn't know where to go on the network to find the Queue.

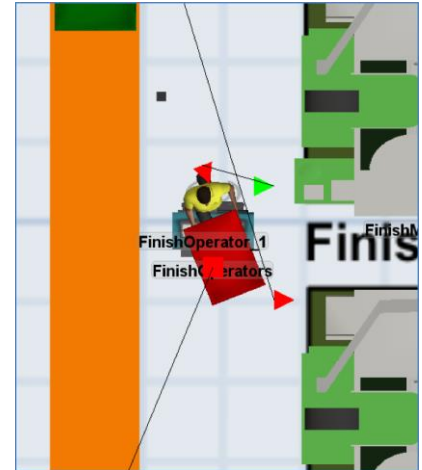


If a Task Executer is not connected to the network, then the TE will not travel on the network, but there will be no error messages. It should be obvious by watching the model that the TE is not constrained by the network paths.

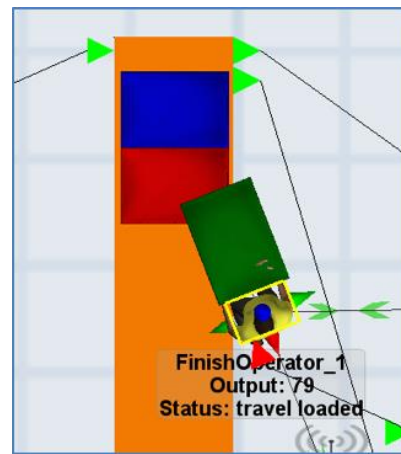
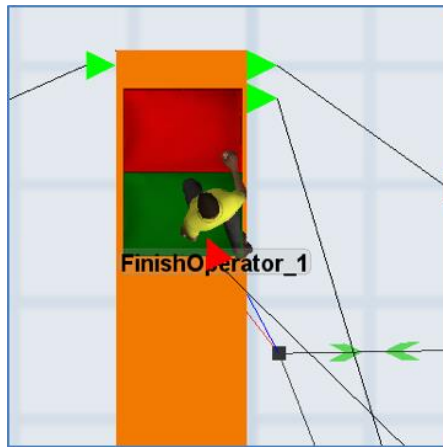
The network can be hidden by right clicking on one of the Nodes and selecting **Network View Mode** from the drop-down menu and then selecting one of the following options:

- **Show All** is the default and what is seen when the networks are created.
- **Edges** show only the connecting lines and not the Nodes.
- **None** shows only one Node, the one selected, and no edges.

The last case is shown in the figure to the right.



A TE will remain on a network path until it gets to a Node that is associated with the task it is executing. At the Node it can either **travel offset** or remain at the Node to execute the task. By default, it travels offset. This is illustrated in the figure to the left below. In this case, the Operator leaves the Node and travels to the object to load the item. If the TE does not travel offset, it performs its task at the Node, as shown in the figure to the right below. In this case, the Operator acquires the container from the Node no matter how far it is from the Node. Whether a TE travels offset or not is a property on the Operator tab.



For the primer example:

- Change the property on the Operator to **Do not travel offsets for load/unload tasks**. This invokes the case shown in the figure to the right above.



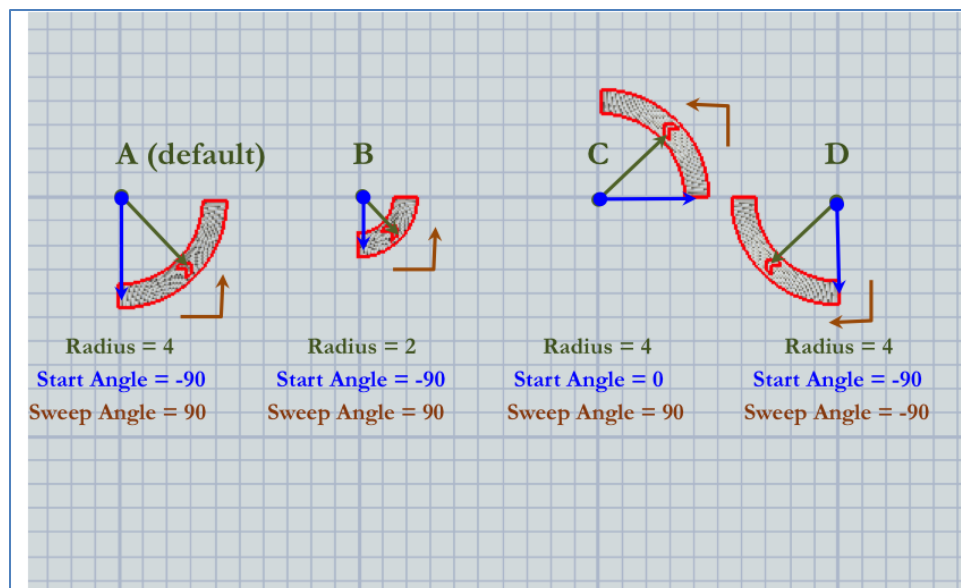
If you haven't already done so, save the model. Recall, it is good practice to save often.

4 CONVEYORS

A common way to move items through a system is via conveyors. Conveyor systems are often complex, but *FlexSim* has extensive capabilities to model such systems. Of course, for this introductory primer, only the basics are considered. Objects associated with conveyors are in a section in the Object Library just below Travel Networks. At this time, the primer considers only the first three conveyor objects: Straight Conveyors, Curved Conveyors, and Join Conveyors. These object can be used to represent either roller or belt conveyors.

Conveyors are much like the Fixed Resources, e.g., Source, Queue, and Processor, with a few key differences.

- Conveyors have two ends, referred to as Start and End, that can be modeled independently. Flow items enter the conveyor object at the Start and leave at the End.
- Conveyors cannot be rotated in the x, y, or z directions as fixed resources can. However, the height (z location) can be changed at either end, resulting in a ramp or elevated conveyor.
- Each conveyor has a direction of travel, indicated by an arrow on the conveyor surface that is visible when the object is selected.
- Curved conveyor shapes are defined by three properties, which are described below and illustrated through a few examples in the figure below. These properties are edited through the Quick Properties interface, rather than double clicking on the object. The properties can also be changed by selecting the conveyor segment and dragging the resizing arrows.
 - **Radius** is the size of the curvature of the conveyor, as measured from the center of a hypothetical circle to the midpoint of the conveyor width.
 - **Start Angle** is the rotation, in degrees, to the location of the Start of the conveyor
 - **Sweep Angle** is the rotation, in degrees, from the Start to the End of the conveyor.



In the figure above,

- Case A shows the default setting; i.e., those of the curved conveyor section when it is dragged out from the Library. The conveyor segment starts 90 degrees below horizontal (Start Angle = -90) and is formed

by sweeping up to the horizontal (Sweep Angle = 90) with a radius of 4 grid units (meters in this model).

- Case B is the same as Case A, but with a smaller radius.
- Case C is the same as Case A but starts at a different location (Start Angle = 0).
- Case D is the same as Case A except it sweeps in the opposite direction (Sweep Angle = -90).

Obtaining the proper shape sometimes involves experimenting with the settings.



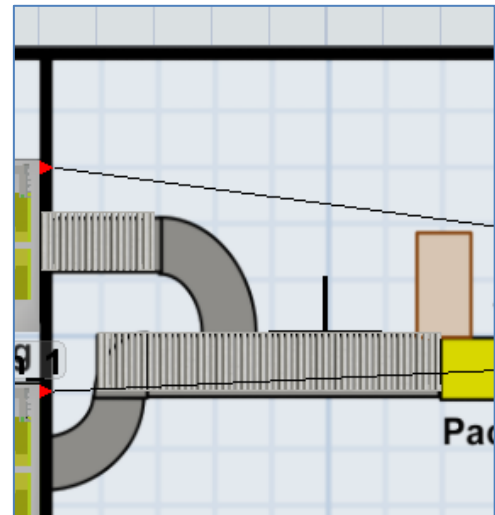
Begin with the basic model from the previous section, named **Primer_2-2**, and **Save As** **Primer_2-3**. Basically make a copy of the model so that it can be customized and the original model is retained.

4.1 Straight and curved conveyor sections

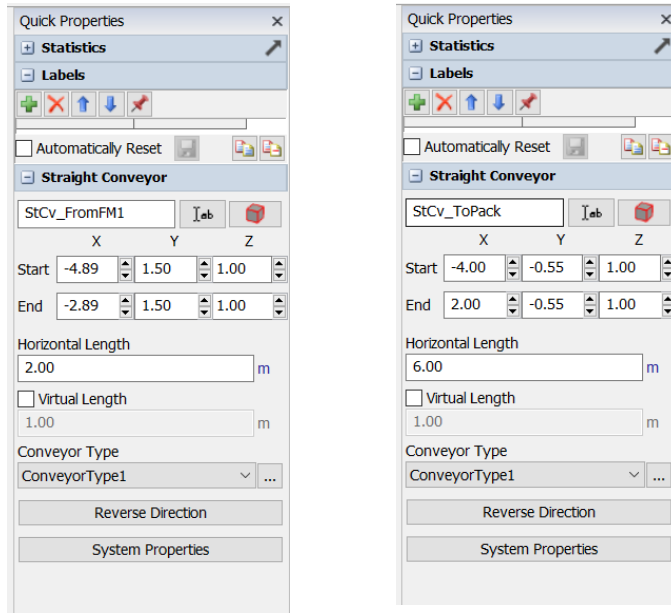
For now, only the conveyors up to the Packing area are modeled. Again, this is an example of the best practice of incremental model building – build small segments, test and validate, and then consider the next aspect of the system.

As described earlier, when precisely placing objects it is best to work in an environment with the following view characteristics:

- 2D rather than 3D view. Right click on the modeling surface, select **View**, and then **Reset View**.
- Flat rather than perspective view. Uncheck the **Perspective Projection** box in Quick Properties.
- Drag out two Straight Conveyor sections.
 - In the Quick Properties window, rename the two sections to **StCv_FromFM1** (straight conveyor from Finish Machine 1) and **StCv_ToPacking** (straight conveyor to the Packing area).
 - Set the first segment's length to **2** and the second to **6** by selecting the conveyor segment and in Quick Properties window change the **Horizontal Length** from the default of 10 (meters in this example).
 - Position both segments on the layout where the conveyors are drawn, as shown in the figure to the right.



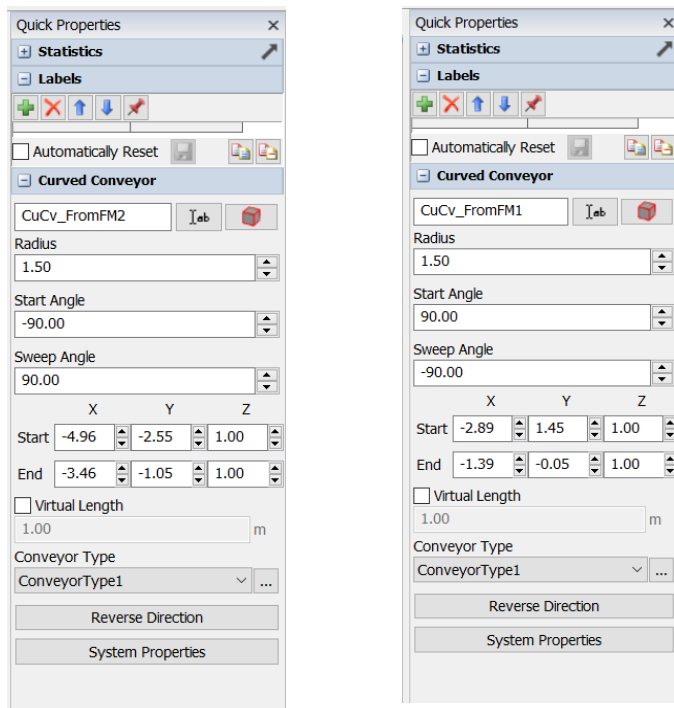
The interfaces for each segment is shown below.



➤ Drag out two Curved Conveyor sections.

- Name one **CuCv_FromFM2** (curved conveyor from Finish Machine 2), set its **Radius** to 1.5, and locate the object in the proper place on the layout. Note and retain the default settings for **Start Angle** and **Sweep Angle**.
- Name the other **CuCv_FromFM1**, set its **Radius** also to 1.5, change the settings for **Start Angle** to 90 and **Sweep Angle** to -90, and locate the object in the proper place on the layout.

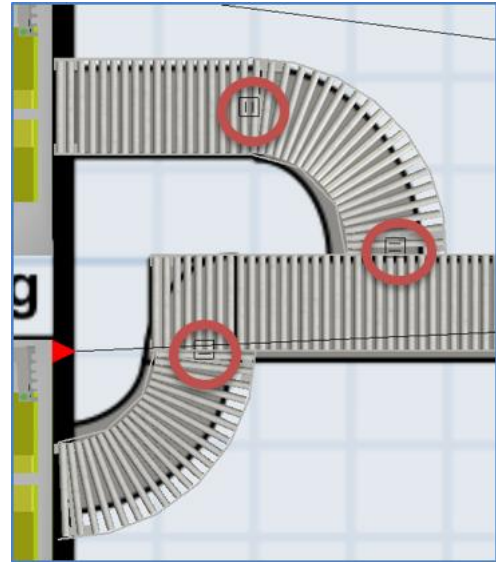
The interfaces for each segment is shown below.



4.2 Connecting conveyor sections

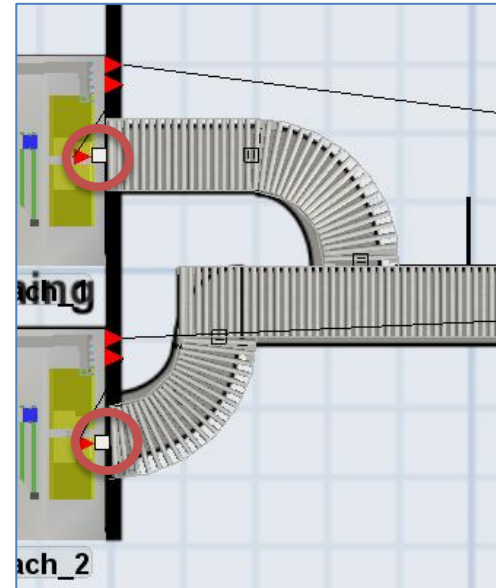
Connecting conveyor sections to each other is different than connecting other *FlexSim* objects. When one conveyor section is selected and dragged close to another section they automatically snap together. As shown in the figure to the right, and highlighted by the circles, a **Transfer** connects the two sections together. The Transfer is the small square that overlaps the two conveyor sections. You may need to zoom in to see the Transfers.

- Check to be sure all segments are connected as shown in the figure.



Conveyor sections are connected to basic *FlexSim* objects in the same way as described throughout the primer, with an A-connection. An Entry Transfer is automatically added when a Fixed Resource object is connected to a conveyor section and an Exit Transfer is automatically added when a conveyor section is connected to a Fixed Resource. In the figure to the right, the Transfers are small white squares at the start of the conveyor sections at the finish machines. They are highlighted by the circles.

- A-connect FinishMachine_1 to StCv_FromFM1
- A-connect FinishMachine_2 to CuCv_FromFM2



Since the conveyors now move the flowitems to the Sink, remove the connections from both finish machines to the Sink.

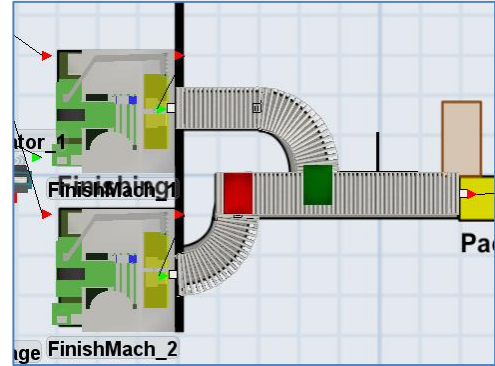
- Q-disconnect the FinishMachine_1 and NextProcess objects. The Q-disconnect is the opposite of an A-connect; therefore, hold down the Q key, select the finish machine and then select the Sink.
- Using the same process as above, Q-disconnect the FinishMachine_2 and NextProcess objects.

Connect the Straight Conveyor StCv_ToPacking to the Sink. Note that connecting the Straight Conveyor to the Sink creates an Exit Transfer at the end of the conveyor.

- A-connect StCv_ToPack to NextProcess, the Sink.

➤ **Reset and Run** the model.

The operation should resemble the figure to the right. Note how the containers move on the conveyor system. As shown in the figure, the containers change orientation because objects are transported based on their orientation when entering a conveyor section. When entering the conveyor system from the finish machines the leading edge of the containers is the short dimension. However, as the containers move onto the straight conveyor that goes to the packing area, the leading edge of the container is now the long dimension - that is the orientation when the items joined the straight section of conveyor from the curved sections. If the orientation should remain the same on all sections of the conveyor, then the Join Conveyors object can be used. This is defined in the next section.



If you haven't already done so, save the model. Recall, it is good practice to save often.



The *Transfers* (between two conveyor sections and between a Conveyor section and a Fixed Resource object) are actually objects as well. They contain advanced properties that can be accessed by double clicking on a transfer.

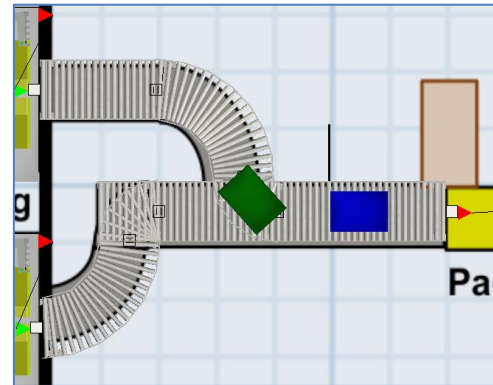
4.3 Join Conveyors object

The Join Conveyor object is more of a tool than an object. It connects two conveyor sections with a curve. In this case, we'll use the Join Conveyor tool for where the curved conveyor from each finish machine joins the main straight conveyor that goes to the packing area.

- Click on the Join Conveyors object in the Library to enter into *conveyor joining mode*. When in conveyor joining mode, the mouse pointer changes to a plus sign with a Join  Conveyors icon next to it. To exit this mode, press the ESC key.
- Click near the center of the first conveyor section to be joined, CuCv_FromFM1, and a yellow line will follow the mouse as it moves, similar to the line that appears when connecting two *FlexSim* objects. Click on the second conveyor section that is to be joined, StCv_ToPack. As a result, a new curved conveyor will appear between the two conveyor sections and two transfer points will also appear on both ends of the new conveyor section.
- Use the ESC key to exit conveyor joining mode.
- Repeat the above steps to join CuCvFromFM2 and StCv_ToPack, where the second finish-machine conveyor joins the straight conveyor.

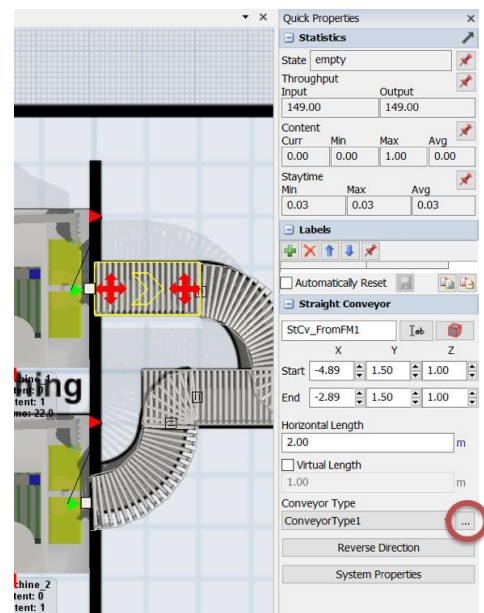
➤ **Reset** and **Run** the model.

The operation should resemble the figure to the right. Note how the containers now maintain the same orientation with the small edge of the container as the leading edge.

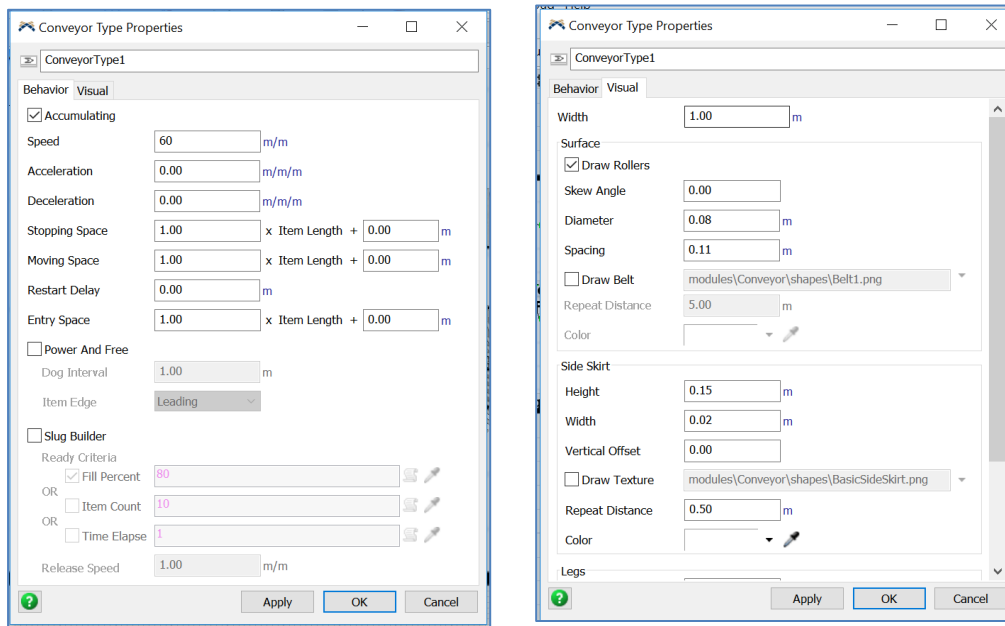


4.4 Conveyor types

Each conveyor segment belongs to a Conveyor Type. By default, all conveyors are of type Conveyor Type 1. The type is accessed through the Quick Properties window. Accessing the type is illustrated in the figure to the right for the straight-conveyor from Finish Machine 1, named StCv_FromFM1. Conveyor Type is accessed through the ... button, just to the right of **Conveyor Type** on the object's Quick Properties window; this is highlighted by the red circle in the figure to the right..



As shown in the figures below, the Conveyor Type Properties controls many aspects of the conveyor's operation. The tab in the left figure, named Behavior, is used to set speed, spacing, etc. The tab shown in the figure to the right, named Visual, is used to set appearance properties.



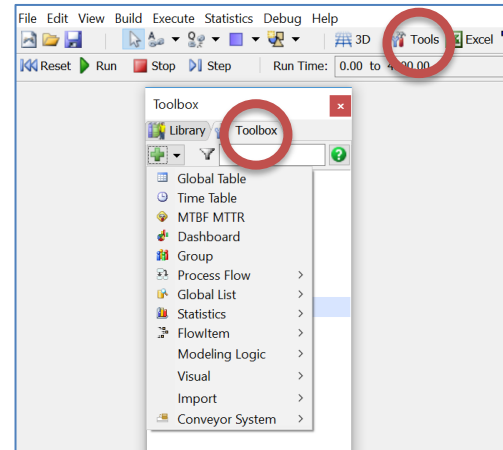
- On the Behavior tab, change the **Speed** from the default of 60.0 to 90 meters per minute. Since all conveyor segments in the model are of Type 1, they all have a speed of 90 meters per minute. To model additional conveyors in the system that have different speeds, create a new type and assign all applicable segments this type value.
- Note, also on the Behavior tab, that by default the conveyor type is accumulating since the **Accumulating** check box is selected. This means that the conveyor acts like a *roller conveyor*. In a roller conveyor, an item travels along the conveyor until the end and then stops if it cannot be removed (downstream object or transport is not available). Subsequent items on the conveyor continue to flow on the conveyor and stop behind the one in front.
 If **Accumulating** is not checked it is a *non-accumulating conveyor* which means it operates like a *belt conveyor*. In a belt conveyor, as in a roller conveyor, an item travels down the conveyor until the end and then stops if it cannot be removed (downstream object or transport is not available). However, on a belt conveyor when one item stops, the belt stops and then all other items on the conveyor stop in their current location.
- Also, note a few of the properties on the Visual tab, albeit none of their values are changed at this time.
 - **Width** is the conveyor's width with a default of 1 grid unit; in this example, this represents 1 meter.
 - The **Surface** section is where the conveyor's rendering properties are set for drawing rollers, belts, or a texture, setting color, etc.



If you haven't already done so, save the model. Recall, it is good practice to save often.

5 GLOBAL TABLES

The primer now switches from *FlexSim* modeling *objects* to *FlexSim* modeling *tools*. As the name suggests, tools are means to support various aspects of simulation modeling and analysis. Tools are accessed through the Toolbox Library. As shown by the circles in the figure to the right, the Toolbox Library is accessed either through: (1) the tab next to the Objects Library or (2) the Tools icon on the Main Menu bar.



Also, shown in the figure to the right, tools are added to a model through the + button in the upper left portion of the interface. Tools include Global Table, Time Table, MTBF/MTTR, etc. These are discussed in subsequent sections of the primer; however, since this is an introduction, not all of the tools are discussed. One tool, Dashboards, was discussed in Section 6.2 of Part 1.



It is good modeling practice to separate data (property values) from objects, especially if the values are to be changed, e.g. when conducting what-if analyses and experimentation.

Tables provide a very convenient means for storing data because during model execution the model can reference a table cell when it needs a parameter value. Tables can also be used to store output from a model. *FlexSim* provides a link to *MSExcel* so that data can be easily exchanged (imported or exported) between a model and *MSExcel* via Global Tables.

Two examples are provided in this section in order to introduce the use of Global Tables. The first example uses a table to store process times for the different types of containers; the second uses tables to store the container product mix, the expected percentage of each type of container.



[section 5.3] Managing data through tables

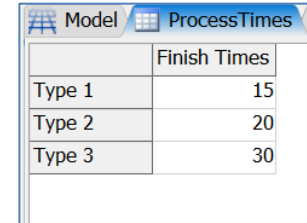


Begin with the basic model from the previous section, named Primer_2-3, and **Save As** Primer_2-4. Basically make a copy of the model so that it can be customized and the original model is retained.

5.1 Using tables to store process times

Process times at the finish machines are stored in a table rather than within each Processor object. For one thing, this facilitates changing parameter values. A value only needs to be changed in the table and is propagated to all objects that reference that table cell.

- Using the green + on the Toolbox, create a Global Table and customize it to reflect the figure to the right.
- In the Quick Properties window, change the name of the table from GlobalTable1 to a more meaningful name, say **ProcessTimes**.
- Also in the Quick Properties window set the number of **Rows** to **3** and leave the default number of **Columns** at **1**.
- Set the cell values as shown - the process times for container types 1 through 3, respectively: 15.0, 20.0, and 30.0 minutes.
- Change shaded “headings” for the rows and columns from the default Row 1, Row 2, ... and Col 1 to the more descriptive names shown in the figure. Note that the headings are just text for information purposes and are not table values. However, they are very helpful in understanding what data tables contain.
- The row heights and column widths can be changed by dragging the lines dividing the cells.

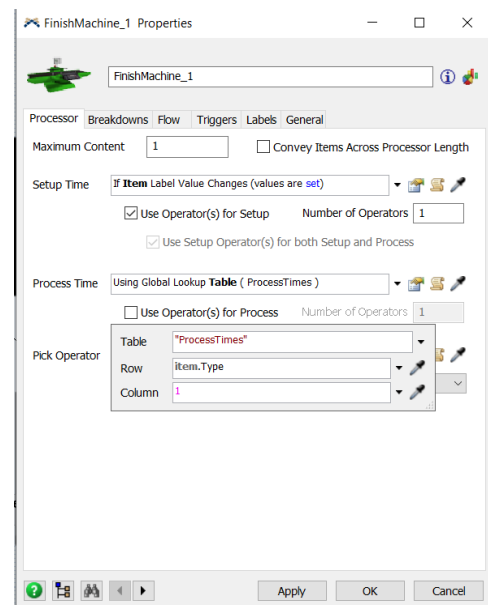


	Finish Times
Type 1	15
Type 2	20
Type 3	30

Note that the table opens in a tabbed window, like the model. The view may be closed by pressing the X in the upper-right corner of the interface. The window can be reopened by double-clicking the table in the Toolbox.

Now the finish machines’ process logic has to be changed in order to read the values from the Global Table.

- Change the Processor’s **Process Time** trigger option from the previous **Values By Case** logic to **By Global Table Lookup** by using the drop-down menu.
 - Complete the property interface as shown in the figure to the right. The **Table** value is **ProcessTimes** - the name of the Global Table created above. It is selected from the drop-down menu options.
- If Global Tables are created before accessing them in the model, then they are available via a drop-down menu. Otherwise, the table name must be typed in with quotation marks around the name. The name must be typed exactly as it is named in the Global Table – names are case sensitive in *FlexSim*. Therefore, it is better to select the name from the drop-down menu.
- Use the default values for the **Row** and **Column** properties. The **Row** value is the value of the item’s (container) label Type. As defined earlier, this is specified in *FlexSim* using dot notation in the format **item.Type**. The finish times are stored in Column **1**.
 - Repeat the above steps for the other finish machine.



In summary, the process time for an item (container in this example) entering the Processor (finishing machine) is determined by a trigger that obtains the value stored in Column 1 and in the row in the Global Table ProcessTimes that corresponds to the value of the item's label Type.

- **Reset** and **Run** the model. Verify that the containers have different finishing times by observing the different speeds that the containers have as they are conveyed across each Processor.



Items can remain static on a Processor while being processed by unchecking **Convey Items Across Processor Length** on the Processor tab. Checking or unchecking the box is for visual purposes only and does not affect the model's behavior; i.e., the processing time is the same whether the item is conveyed across the object or not.



If you haven't already done so, save the model. Recall, it is good practice to save often.

- Uncheck the **Convey Items Across Processor Length** box on both finish machine's Processor tab.

5.2 Using tables to store product mix percentages

A second example of using a Global Table to store data involves the product mix of containers. Recall that in the current model, as defined in Section 5.1 of Part 1, the container type is randomly generated when it is created at the Source ContainersArrive. Recall also that it is implemented by using the *FlexSim's* discrete-uniform distribution command `duniform(1, 3, getstream(current))`; i.e., each container's Type is equally likely between 1 and 3.

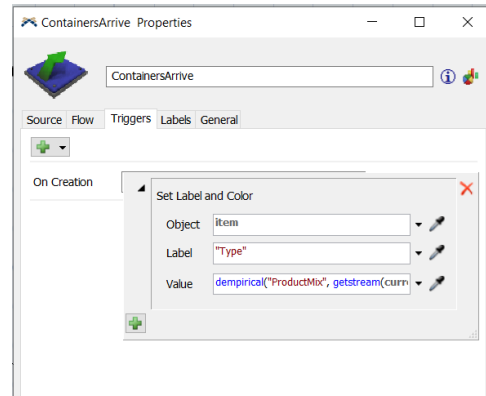
However, the product mix of container types is most likely not equal. Even if this were the case now, it would likely change in the future. A more general product mix is implemented by storing the information in a Global Table and using a different *FlexSim* command at the Source.

- In the same manner as described above, create a Global Table, name it **ProductMix**, and set the number of **Rows** to **3** and the number of **Columns** to **2**.
- As shown in the figure to the right, label the header rows and columns. Again, the headers are used to clarify what information is stored in the table. The rows correspond to the product types.
- Set the cell values as shown. Column 1 is the percentage of that type and Column 2 is the value of the type. In this case, 20% are Type = 1, 30% Type = 2, and 50% Type = 3; i.e., in the long run, half of the containers will be Type 3 (blue). *The percentages in Column 1 of the table must add to 100%.*

ProductMix		
	Percent	Type
Type 1	20	1
Type 2	30	2
Type 3	50	3

Now the Source's logic has to be changed in order to randomly generate a Type based on the percentages stored in a Global Table. The change is shown in the figure to the right and described below.

- Select the Source's existing **OnCreation** trigger option **Set Label and Color**.
- In the resulting interface, no changes need to be made to the **Object** and **Label** properties.
- Replace the **Value** property, `duniform(1, 3, getstream(current))`. To do so, select the **Statistical Distributions** option from the drop-down menu option and then select **dempirical** from the list of distributions.
- Modify the **dempirical** command in the **Value** box to the following. (Change the parameter "MyTable" to "ProductMix")
`dempirical ("ProductMix", getstream(current))`



The discrete-empirical command randomly generates a number between 0 and 100 percent and uses that number to look up a Type value in the Global Table, in this example ProcessTimes.



If you haven't already done so, save the model. Recall, it is good practice to save often.

6 DOWNTIME

In *FlexSim*, there are two means for making resources not available, i.e. incurring downtime – the Time Table tool and the MTBF/MTTR tool. Both are accessed through the Toolbox Library. A powerful feature of *FlexSim* is the management of downtimes, especially multiple types of downtimes. As such:

- Any object can be subjected to downtime.
- Any object can be subjected to multiple types of downtimes, oftentimes referred to as *competing downtimes*.
- Multiple objects can follow the same downtime process.



Begin with the basic model from the previous section, named Primer_2-4, and **Save As** Primer_2-5. Basically make a copy of the model so that it can be customized and the original model is retained.

6.1 Time Tables

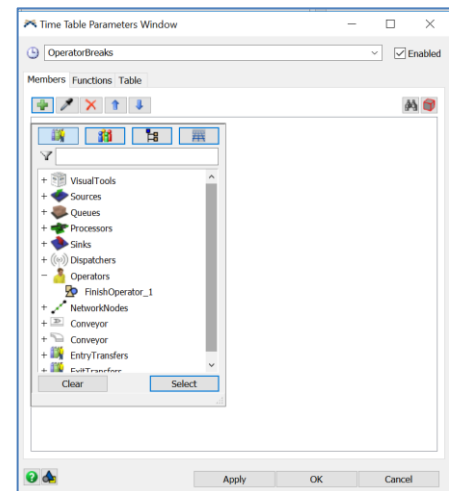
Time tables are used to model planned downtimes; i.e., ones that occur on a known and recurring basis, such as breaks, shift schedules.



[section 7-1] Establishing Time Tables

In this example, the finish operator takes a 15-minute break two hours into an 8-hour shift, a 30-minute lunch break four hours into the shift, and another 15-minute break six hours into the shift. It is implemented via the Time Table tool and its three tabs – Table, Functions, and Members. The property settings for this tool are explained below.

- In the Toolbox, press the + button and select Time Table from the drop-down menu.
- Change the name from TimeTable1 to **OperatorBreaks**.
- The Members tab is used to assign this downtime pattern to one or more objects. Using the + button, select **FinishOperator_1** from the **Operators** object category and press the **Select** button. The selection is shown in the figure to the right.

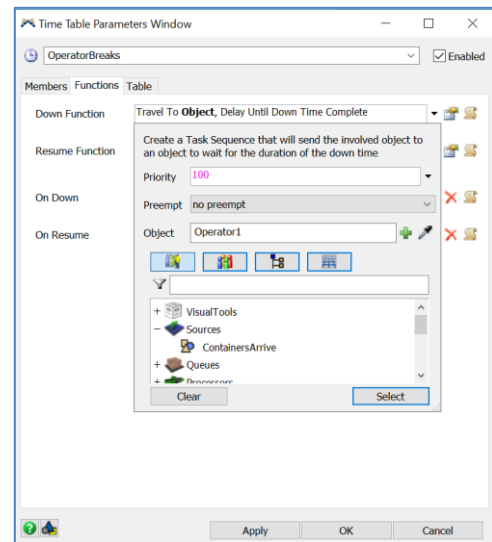


The Functions tab is used to control what happens when a resource goes down (becomes unavailable) and then resumes. By default, *FlexSim* stops the object at the appropriate time in the simulation and the object remains down for the prescribed time. In this example, the following behavior is added to the default.

The default **Down Function** stops the resource wherever it is and whatever it is doing and it remains at that location for the prescribed duration. However, in this example, the Operator takes a break at a specified location, at the Source (ContainersArrive). The prescribed delay starts when the Operator reaches the specified location.

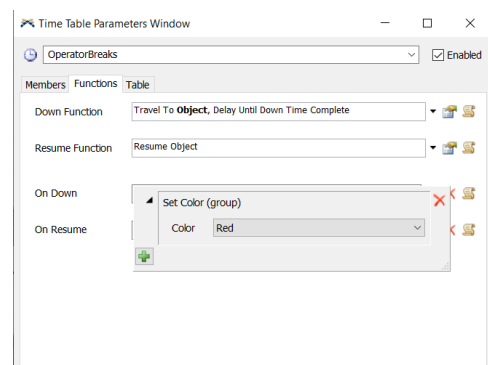
To implement this, complete the following steps and refer to the figure to the right.

- Change the **Down Function** property from **Stop Object** to **Travel To Object, Delay Until Down Time Complete** by selecting the option from the drop-down menu.
- In the resulting dialog set the **Object** property to **ContainersArrive** using the + button and selecting the object from the **Source** category. Retain the default values for the **Priority** and **Preempt** properties.



In order to provide a visual cue that the operator is not available, change the object's color during the break period. To implement this, complete the following steps and refer to the figure to the right

- Change the **On Down** option to **Set Color (Group)** and the **Color** property value to **Red** (the default). This changes the operator's color to red when on break.
- Reset the operator's color after the downtime is concluded in a similar manner. Change the **On Resume** property to **Set Color (Group)** and then the **Color** property to **Yellow**. from the drop-down list of colors. This changes the operator's color back to its default color, yellow, when the operator is available for work.

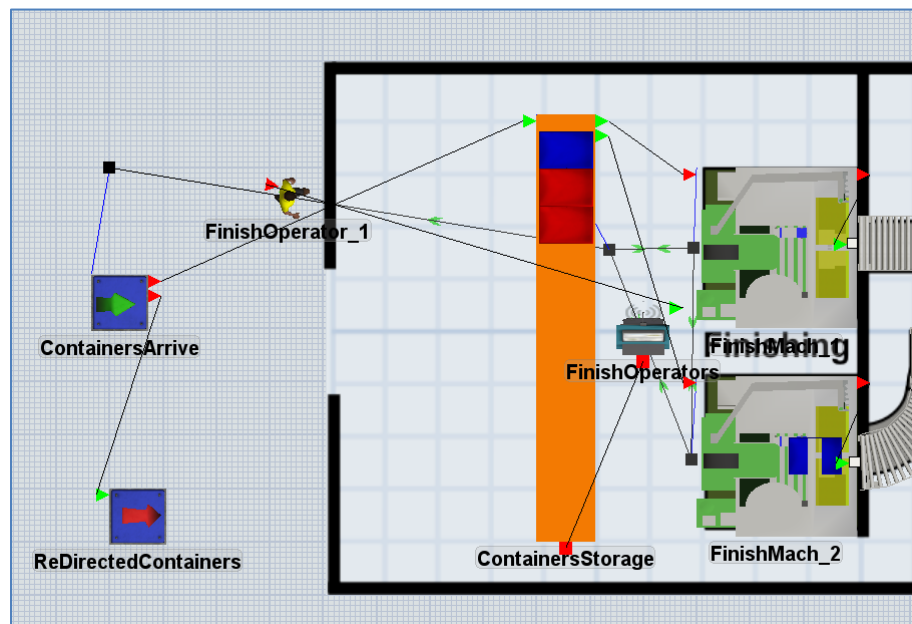


As shown in the figure to the right, the Table tab is used to set the times the resource becomes unavailable (**Time**) and for how long (**Duration**). In this case, there are three downtimes per shift and all values are in minutes, the time unit specified for the model.

- Change the **Mode** from **Date Based** to **Custom Repeat**.
- In the field next to the **Custom Repeat** value, set the repeat time to 480. The pattern entered into the table will now repeat every 480 minutes of simulated time. This corresponds to an 8-hour shift. This is handy when simulating multiple shifts so that the downtimes for each shift do not have to be explicitly included in the time table; the provided pattern just repeats for as long as the simulation runs.
- Since there are three downtimes per shift, set **Rows** to 3.
- The first downtime is a 15-minutes break that occurs two hours into the shift; therefore, set **Time** to 120 minutes and **Duration** to 15 for the first row.
- Change the **State** parameter for each of the three rows from the default 12 to 35. Clicking on the cell results in a list of states; select 35. The state value 12 means “scheduled down” and state value 35 means “on break.” These are two of the 50 possible states defined in *FlexSim*. The **State** property is used for reporting the resource’s utilization. Keep the default values for the **Profile** and **DownBehavior** columns.
- The second downtime is a 30-minute lunch break that occurs four hours into the shift; therefore, set **Time** to 240 minutes and **Duration** to 30 for the second row.
- The third downtime is another 15-minutes break that occurs six hours into the shift; therefore, set **Time** to 360 minutes and **Duration** to 15 for the third row.

Time	State	Duration	Profile	DownBehavior
120	35	15	0	0x0
240	35	30	0	0x0
360	35	15	0	0x0

Since the Operator travels to the Source for breaks, and since the Operator is constrained to the network, the Source needs to be placed on the network; i.e., associate it with a new network node that is connected to the other nodes. This is shown in the figure below. While the implementation steps are the same as described in Section 3.3, they are repeated again below.



- Select a Network Node from the Object Library and drag and drop it near the Sink, ContainersArrive.
 - Name the node **nn_OperatorBreaks**.
 - A-connect the node to the previously-constructed network for the Operator, e.g. to the node near the Queue, nn_ContainerStorage.
 - A-connect the Source object to the new node.
-
- **Reset** and **Run** the model. Verify that the operator travels to the Source at the prescribed times (120, 240, 360, etc.).



If you haven't already done so, save the model. Recall, it is good practice to save often.

6.2 Reliability

Reliability refers to an object being able to perform its required functions for a specified time. It is an important contributor to overall system performance. Two key factors are used to specify reliability, *Mean Time Between Failures* (MTBF) and *Mean Time To Repair* (MTTR). The former is the up time, or time an object operates, expressed as an average or mean value; the later is the time it takes to get a down object back to its operational state, again expressed as an average or mean value.



[section 7-3 through 7-6] Reliability, Estimating MTBF and MTTR, Simulating machine failures, and Setting MTBF and MTTR in a simulation.

In this example, each finish machine is subject to two types of downtimes. Thus, the model has competing downtimes that must be managed, which *FlexSim* does very well. For example, in most cases if a resource is already down or not available, it cannot incur another type of downtime.

6.3 Constant MTTF/MTTR; MTBF based on clock time; no resource for repair

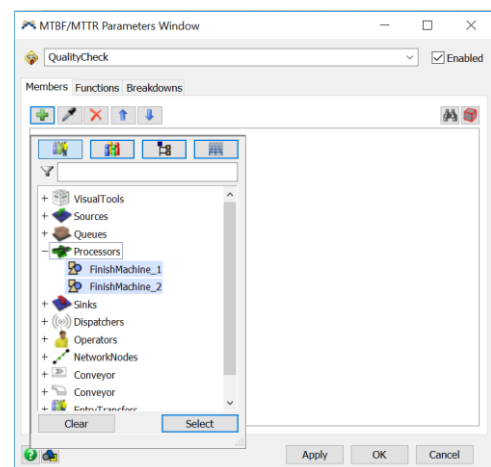
The first type of downtime on the finish machines is a quality check that occurs every 10 minutes for 15 seconds; i.e., MTBF is a constant 10 minutes and MTTR is a constant 15 seconds. The downtime occurs regardless of the machine's current state and how much the machine has been used; i.e., MTBF depends on clock time only. No other resource is needed; i.e., the machine is just down for the duration of the self check.

- In the Toolbox, press the + button and select the **MTBF MTTR** tool from the drop-down menu.
- In the tool's Parameter Window change its name from MTBFMTTR1 to **QualityCheck**.

The MTBF MTTR tool contains three tabs – Members, Functions, and Breakdowns. The settings for the quality check downtime are explained below.

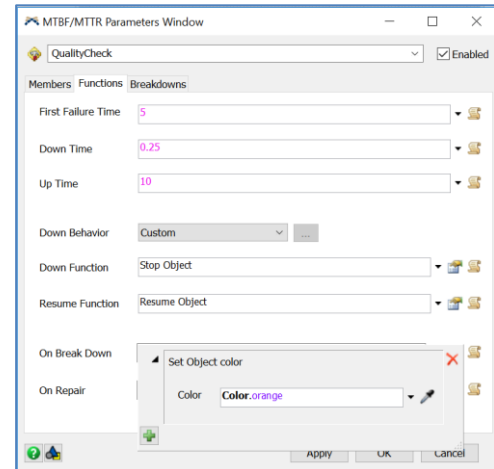
The Members tab is used to assign this downtime pattern to one or more objects; i.e., the downtime behavior defined on the Functions and Breakdowns tabs will be applied to all objects in the Members tab.

- In this case, using the + button, select both finish machines, **FinishMach_1** and **FinishMach_2** from the **Processors** category. This is shown in the figure to the right. Alternatively, if the category **Processors** is selected, all Processors are selected.



The Functions tab defines how often downtimes occur and their durations. In this case, all values are deterministic or constant; thus, the default probability distributions will be replaced by constants, as shown in the figure to the right..

- Change the **First Failure Time** property from the exponential probability distribution to a constant **5** minutes. This assumes the simulation starts halfway between downtimes.
- Change the **Down Time** property from the exponential probability distribution to a constant 15 seconds (**0.25** minutes).
- Change the **Up Time** property from the exponential probability distribution to a constant **10** minutes. This is the time between failures or between down states,.



The default values are used for the **Down Behavior**, **Down Function**, and **Resume Function**; i.e., when the object experiences a downtime it is stopped for the duration of the downtime and then restarted. No resource is needed during the downtime; the machine processes the downtime itself.

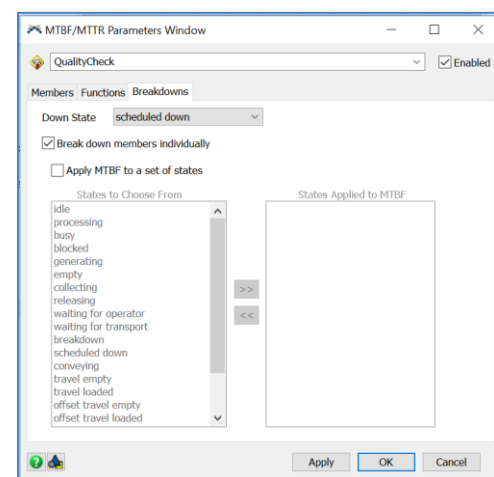
The **On Break Down** and **On Repair** triggers are used to change the color of the object when it is down. In this case, the object is orange when it is down for a quality check and back to green when not being checked.

- The **On Break Down** trigger sets the object's color to orange indicating that it is down. As shown in the figure above, the **Set Color (individual)** option is selected from the drop-down menu and the color is set to **Color.orange**.
- Similarly, the **On Repair** trigger sets the object's color back to green indicating it is not down. The **Set Color (individual)** option is selected from the drop-down menu and then **Color.green** is selected.

Now consider the third tab, Breakdowns.

For clarity, it is best to differentiate the downtime state for failures, which is a “breakdown,” as described in the next section, and the planned downtime for the quality checks. For the check quality case, the time between failures (downtimes) is based on the simulation clock and not on any of the object's states; i.e., the machine will be unavailable for 15 seconds every 10 minutes, regardless if it is processing or not, in order to upload data. Therefore, the default is used in this case. This will change in the failure downtime described in the next section.

- On the Breakdowns tab, as shown in the figure to the right, change the **Down State** from the default, **breakdown**, to **scheduled down**, using the list of states on the menu. This state is more descriptive of the behavior since the quality check is a scheduled downtime and not regarded as a failure.



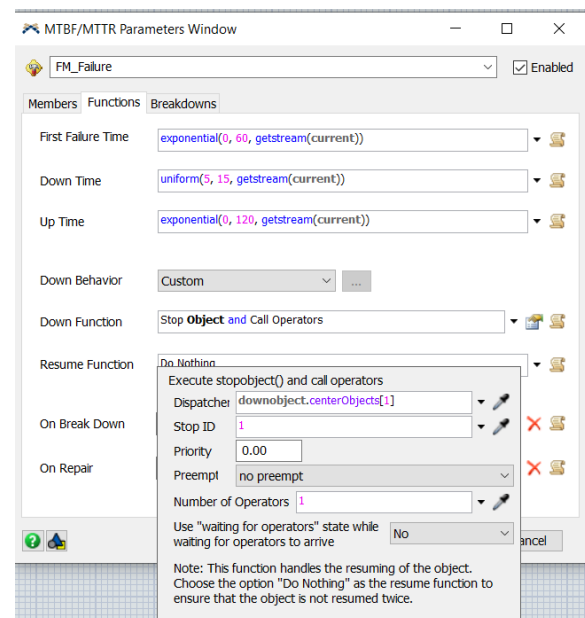
6.4 Random MTTF/MTTR; MTBF based on system states; resource for repair

The second type of downtime on the finish machines is a random machine failure. The times between failures and the times to repair are both considered random variables or probabilistic and are based on probability distributions. For this type of downtime, a resource – the finish operator – is needed to perform the repair.

- In the Toolbox, press the + button and again select the **MTBF MTTR** tool from the drop-down menu.
- In its parameter window, change its name to **FM_Failure**.
- On the Members tab, just as was done with the QualityCheck downtime described above, use the + button and select both finish machines, **FinishMach_1** and **FinishMach_2**, from the **Processors** category.

The Functions tab defines how often downtimes occur and their duration. It also defines what actions occur in the simulation whenever a downtime occurs. The Functions tab for this example is shown in the figure to the right and explained below.

In this case, all of the times are probabilistic and are specified in terms of the default probability distribution. For the times between failures (**First Failure Time** and **Up Time**), the exponential distribution is the default. The only parameter required for this distribution is its mean value. For the repair times, the default distribution is the uniform. Two parameters are required for the uniform distribution, the lowest and highest values, i.e., the shortest and longest possible repair times. All repair times are assumed equally likely between these two limits.



- Change the **First Failure Time**'s second parameter in the exponential distribution from the default value of 1000 to **60** minutes. This is the distribution's mean value and assumes the simulation started halfway between downtimes.
- Change the **Down Time**'s parameters from the default values of 50 and 100 to **5** and **15**. This assumes downtime is uniformly distributed between 5 and 15 minutes.
- The **Up Time** property is the time between failures or time between down states, It is assumed to be exponentially distributed with a mean of two hours (120 minutes). Therefore, change its second parameter, the mean, from the default value of 1000 to **120**.
- Change the **Down Function**, from the default **Stop Object** to **Stop Object and Call Operators** via the property's drop-down menu. The resulting interface is shown in the figure above. All default values are used. This selection, in addition to stopping the object during a downtime, calls a finish operator to perform the repair. The default calls the object that is connected to the object's center port; in this case, it is the Dispatcher.
- According to the note at the bottom of the interface for the **Down Function** property, the **Resume Function** value is to be set to **Do Nothing** via the drop-down menu. This is because the logic for resuming is managed in the down function.

As with the QualityCheck downtime, the **On Break Down** and **On Repair** triggers are used to change the color of the object when it is down.

- For the **On Break Down** trigger, as with the QualityCheck downtime, select **Set Color (individual)** option. However, for failures the object is colored red when it is down; therefore, choose the **Color.red** option.
- For the **On Repair** trigger, as with the QualityCheck downtime, select **Set Color (individual)** option and then set the color back to green using the **Color.green** option.

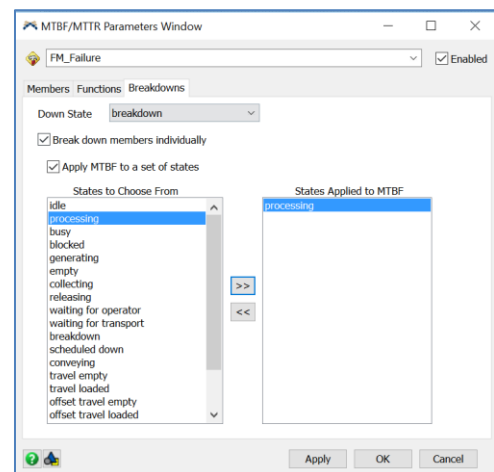


It is important to use the correct distribution since the choice can significantly impact system performance and simulation results. *FlexSim* includes many types of probability distributions. Discussion of how to select the distribution is beyond the scope of this introductory primer.

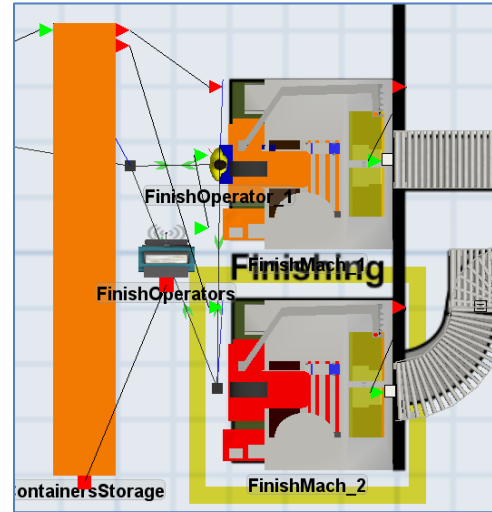
In this example the time between failures (downtimes) is based on the state of the object and not the simulation clock. The finishing machines only accrue downtime when they are running or processing. For example, if the time to the next failure on a machine is two hours and the machine operates only 50% of the time, then the next downtime will occur in four hours of simulation time. Also, a failure will occur only when the object is in one of the selected states. For example, a failure will occur only when a machine is processing and not when it is idle.

The above situation is defined on the Breakdowns tab and is shown in the figure to the right.

- Check the box **Apply MTBF to a set of states**.
- Move states from the **States to Choose From** list to **States Applied to MTBF** using the >> move button. In this case, select only the **processing** state.
- The type of state can be changed in the **Down State** drop-down menu. However, the default **breakdown** is used in this example.



The figure to the right shows the model running when both finish machines are experiencing downtime, albeit different types. FinishMach_1 is undergoing a quality check and thus is colored orange; it is in the scheduled down state. FinishMach_2 has failed and is in the breakdown state, as indicated by its red color.



If you haven't already done so, save the model. Recall, it is good practice to save often.

6.5 Charting the effect of downtime

One way to visualize the effect of downtimes on system performance, is to create a dashboard of charts that show the utilization of the finish machines and the finish operator.

- From the Main Toolbar select **Dashboard** and then **Add a Dashboard**. Alternatively, click the green + on the Toolbox Library and select **Dashboard**.
- Name the new dashboard, **Utilizations**.

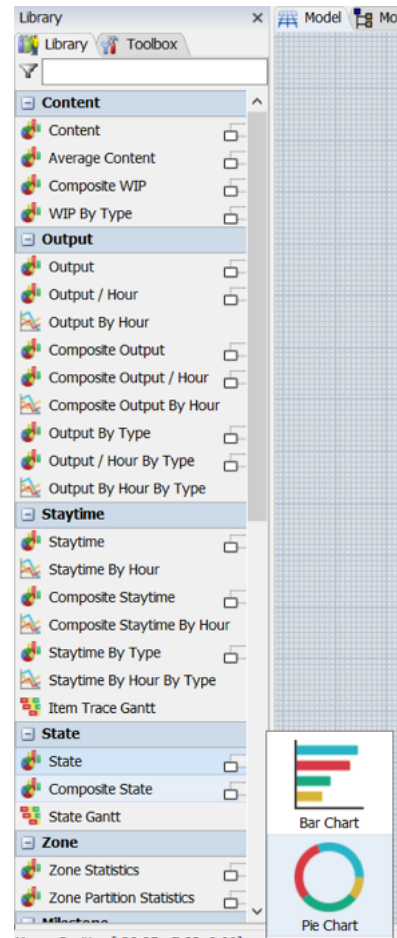
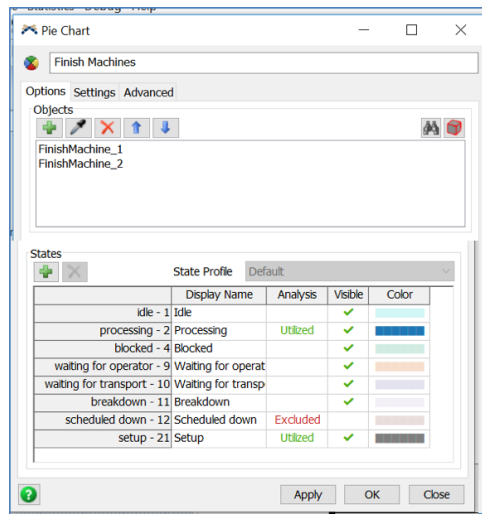
Pie chart for finish machine utilization

Develop a pie chart of the various states of the finish machines to assess their utilization.

- As shown in the figure to the right, select the **State** chart type in the **State** section of the Dashboard Library; then, select **Pie Chart** from the pop-up menu.
- Drag the selection to the dashboard workspace.

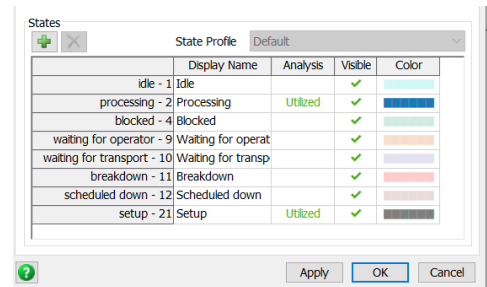
Change the property values as follows and to those shown in the figure below.

- Change the name of the chart from State Pie to **Finish Machines**.
- In the **Objects** section of the Options tab, click the + button and choose the **Select Objects** option. Then, select the category **Processors** and press **Select**. As a result, both FinishMach_1 and FinishMach_2 should appear in the **Objects** section of the interface.



The States section for the Pie Chart shown above contains the default options. However, for this example, the values will be changed to those shown in the figure to the right, both for the state **scheduled down – 12** and **breakdown – 11**. The steps for making the changes are described below.

- Click the cell in the **Analysis** column and **scheduled down – 12** row, currently displaying **Excluded**. This will change the cell to a blank, which means that the state percentage will now be considered “not utilized” on the chart.
- Since the default color of the pie slices for breakdown and scheduled down states are quite similar, they may be hard to differentiate on the chart. Therefore, click the cell in the **Color** column for the **breakdown** row, then select red from the color palette.
- Be sure the **Visible** column has a green check for all of the states. If not, click the blank space in the **Visible** column for that state. The green check means that the state percentage will be displayed on the chart.

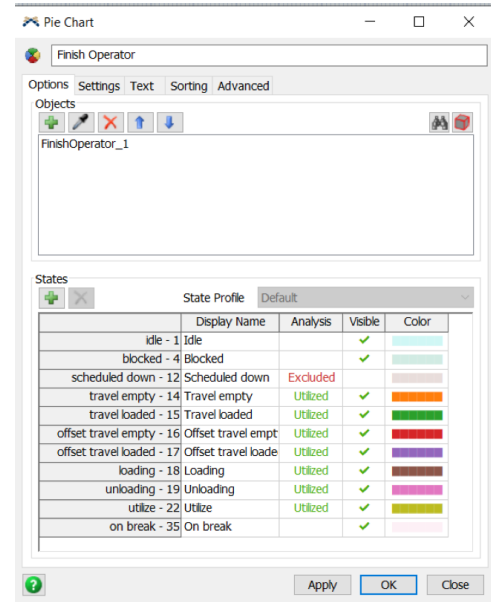


Pie chart for finish operator utilization

Develop another pie chart of the various states of the finish operator to assess its utilization. The process is very similar to the one used for the finish machines.

- Select the **State** chart type in the **State** section of the Dashboard Library; then, select **Pie Chart** from the pop-up menu. Drag the selection from the library to the dashboard workspace. Change the property values as shown in the figure to the right and described below.
- Change the name of the chart to **Finish Operator**.
- In the **Objects** section of the Options tab, click the + button and choose the **Select Objects** option. Then, select the **FinishOperator_1** object in the Operators section of the list and press **Select**.
- For this chart, all of the defaults will be used in the **States** section.

Note the list of states included in this chart is much longer than the previous utilization chart because the Operator object will be in different states than the Processor during the simulation. The percentage utilization for the Operator includes all of the travel-time tasks (empty and loaded) as well as the time spent loading and unloading items, performing setup, etc. Note the on break task is included in the states list, but is not one where the Operator is considered utilized.



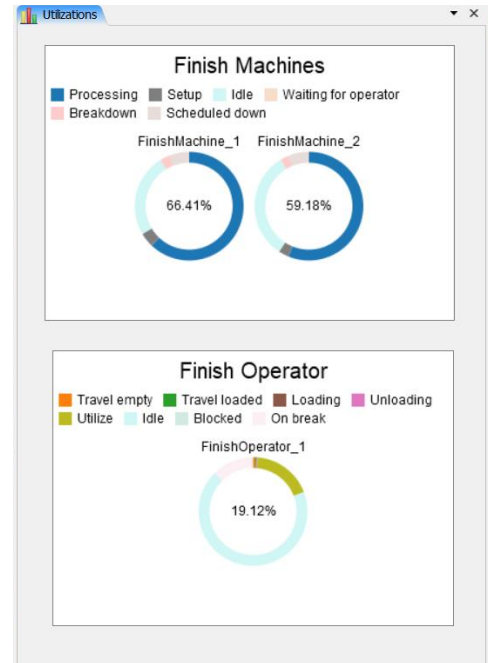
- **Reset** and **Run** the model run for 80 hours (4800 minutes).
The results should be similar to that shown in the figure to the right.

Based on this simulation run, the two finish machines are each busy about 66% and 59% of the time, respectively; or, about 62% overall. These percentages include the times the Processors are in the processing and setup states (overall about 59% and 3%, respectively), as defined in the States section on the Options tab of the Finish Machines' pie chart.

The pie chart also shows the small percentage of the time that they were in the breakdown (about 2%) and scheduled down (about 6%) states. The remainder of the time, the idle state, is when the machines were ready and available, but had nothing to process; in this case, that is about 30% overall.

The utilization of the finish operator, the time the operator is busy, is about 19%. This includes traveling both loaded with containers and unloaded, loading and unloading containers, performing setup operations (part of the Utilize state on the chart), and repairing the finish machines when they breakdown (the other part of the Utilize state). In addition, the operator is in the On-break state about 13% of the time. The remaining 68% of the time the operator is in the idle state, ready and available, but has no tasks to perform.

Note that the exact percentages for each piece of the pie is displayed when the cursor hovers over that area of the chart.



If you haven't already done so, save the model. Recall, it is good practice to save often.